PAI-3 VULNAWEB. Auditoría de seguridad en sistemas informáticos y análisis de vulnerabilidades en una universidad pública

Grupo: Security Team 5

Integrantes: Miguel Ridao y Tomás Gutiérrez

Fecha: 10 de noviembre de 2026

Índice

Resumen ejecutivo 2
1. Auditoría de sistemas informáticos 2
    1.1 Descripción del sistema analizado y herramientas de análisis 2
    1.2 Plan de pruebas y resultados 3
    1.3 Conclusiones 4
2. Análisis y bastionado de navegador web 5
    2.1 Descripción del sistema analizado 5
    2.2 Plan de pruebas y resultados 6
    2.3 Conclusiones 6
3. Auditoría de sistema móvil 6
    3.1 Descripción del sistema analizado y herramientas de análisis 6
    3.2 Plan de pruebas y resultados 6
    3.3 Conclusiones 8
4. Auditoría de sistema web 8
    4.1 Descripción del sistema web y herramientas de análisis 8
    4.2 Plan de pruebas y resultados 8
    4.3 Plan de mitigación y demostración (antes/después) 10
    4.4 Conclusiones 11
Referencias 12
Anexos 13
    Anexo A. Capturas 13
    Anexo B. Ficheros de evidencias 28
    Anexo C. Sugerencias de Lynis que no se aplicaron 29
    Anexo D. Correspondencia con los estándares 29

Resumen ejecutivo

- Sistemas: con **hardening.sh**, el hardening index de Lynis sube de 64 a 73 en Ubuntu, de 64 a 71 en Debian y de 64 a 73 en Kali, los tres por encima del 70 exigido; la mejora viene sobre todo de añadir detección y auditoría (rkhunter, fail2ban, AIDE, auditd).
- Navegador: una política corporativa de Firefox, desplegada por script, fuerza HTTPS y DNS cifrado, impide guardar contraseñas y sube la protección contra el rastreo de "parcial" a "fuerte" en Cover Your Tracks.
- Móvil: los 2 Pixel auditados no tenían root ni malware, pero fallaban en puntos de higiene de la política (Bluetooth encendido en uno; Chrome sin actualizar, Wi-Fi abiertas u ocultas guardadas y apps de terceros con permiso para leer las descargas en el otro; y en los dos, apps de terceros con conexiones abiertas en segundo plano), corregidos y verificados después.
- Web: de los 6 endpoints de escritura probados, 5 aceptan peticiones sin ninguna autenticación, y el sexto usa un secreto JWT hardcodeado que se recupera en segundos con un diccionario dirigido; tras la corrección, los 6 rechazan las peticiones sin un token válido.

El trabajo sigue el esquema de NIST SP 800-115 (auditar, corregir y volver a auditar) en las cuatro partes, y la correspondencia de cada parte con los controles de CIS Benchmarks, OWASP Top 10 y el ENS en categoría Alta está en el Anexo D.

1. Auditoría de sistemas informáticos

1.1 Descripción del sistema analizado y herramientas de análisis

Se han auditado 3 equipos, cada uno con una distribución Linux distinta: dos máquinas virtuales instaladas desde la ISO oficial de su última versión estable, Ubuntu 24.04.5 LTS (última LTS) y Debian 13, y un portátil físico de uso diario con Kali Linux Rolling. Usar tres distribuciones permite comprobar que las medidas de bastionado son portables. Antes de bastionar se tomó un *snapshot* de cada VM. En el portátil no hay snapshot, así que antes de empezar se dejó el sistema sin ninguna medida aplicada, devolviendo cada fichero que toca el script a su contenido original (comprobado con el md5 que guarda el paquete), y se reinició para descargar los parámetros del kernel.

La auditoría se hace con Lynis (perfil por defecto), que revisa cientos de controles y devuelve un *hardening index* de 0 a 100. Se ejecuta antes y después del bastionado para medir la mejora. El bastionado se aplica con el script propio **sistemas/hardening.sh**, idempotente y organizado por grupos que corresponden a avisos concretos de Lynis. Las tres distribuciones usan **apt** (Kali está basada en Debian), así que el mismo script sirve sin cambios en las tres.

Herramientas y medidas que aplica hardening.sh

Grupo	Herramienta / cambio	Cómo protege	Test de Lynis
firewall	**UFW**: deniega todo lo entrante, permite lo saliente	Cierra los puertos no usados; reduce la superficie de ataque por red	FIRE-4590
passwords	**libpam-pwquality** (mín. 12 caracteres, con dígito y mayúscula), caducidad 90 días y **UMASK 027** en **login.defs**	Contraseñas resistentes a fuerza bruta y archivos nuevos no legibles por otros usuarios	AUTH-9262, AUTH-9286, AUTH-9328
banner	Aviso legal en **/etc/issue** y **/etc/issue.net**	Advierte de acceso restringido y registro de actividad (valor legal/disuasorio)	BANN-7126, BANN-7130
usb	Bloqueo de los módulos **usb-storage** y **firewire**	Impide sacar datos o meter malware con memorias USB	USB-1000, STRG-1846
kernel	**sysctl** (**sysctl-hardening.conf**): anti-spoofing, sin redirecciones ICMP, SYN cookies, ASLR completo, ocultar punteros y **dmesg**	Protege la pila de red (spoofing, SYN flood, MITM) y dificulta explotar el kernel	KRNL-6000
coredumps	*** hard core 0** en **limits.conf**	Evita volcados de memoria que podrían contener contraseñas o claves	KRNL-5820
herramientas	**auditd** y **acct** (auditoría y registro de comandos), **sysstat** (estadísticas), **rkhunter** (detección de rootkits), **fail2ban** (bloquea IPs tras intentos fallidos), **AIDE** (integridad de ficheros), **apt-show-versions** (control de parches)	Añade detección y trazabilidad: si hay un ataque, se detecta y queda registrado	ACCT-*, HRDN-7230, FINT-4350, DEB-0880, PKGS-7394

Por qué sube el índice de Lynis: el *hardening index* se calcula con los controles que superan la comprobación frente al total evaluado. Cada grupo del script resuelve una o varias sugerencias concretas del informe inicial de Lynis (columna derecha), así que cada control corregido suma puntos. Se aplicó por grupos, repitiendo Lynis entre ellos, para ver qué aporta cada medida (figuras 1 a 9 del anexo).

1.2 Plan de pruebas y resultados

Equipos. Pai-ubuntu (Ubuntu 24.04.5 LTS) y **pai-debian** (Debian 13), auditados con Lynis 3.1.8 el 2-nov-2026, y **RIDAO** (Kali Linux Rolling, kernel 7.1.5, portátil físico), auditado con Lynis 3.1.7 el 9-nov-2026.

Procedimiento (en cada equipo, siguiendo el esquema de NIST SP 800-115: auditar, corregir, volver a auditar):

1. **sudo ./run_lynis.sh antes**: auditoría inicial.
2. **sudo ./hardening.sh firewall** y nueva pasada de Lynis, para medir esta medida por separado. Esta pasada se guardó también con la etiqueta **antes** y sobrescribió el log inicial, así que el índice inicial (64) queda demostrado con las capturas de las figuras 1 y 4, y el log **antes** corresponde al estado tras el firewall. En Kali esta pasada se guardó con la etiqueta **firewall**, así que se conservan los tres logs.
3. **sudo ./hardening.sh**: el resto de grupos.
4. **sudo ./run_lynis.sh despues**: auditoría final.

Resultados:

Equipo	Índice inicial	Tras firewall	Índice final	Puntos Lynis (tras firewall → final)
Ubuntu 24.04.5	64	66	**73**	206/312 → 242/330
Debian 13	64	65	**71**	214/327 → 246/342
Kali Rolling	64	65	**73**	182/278 → 212/288

Los tres equipos superan el objetivo de ≥ 70. Lynis ejecuta algunos tests más en la pasada final (269 → 281 en Ubuntu, 278 → 287 en Debian, 263 → 269 en Kali), porque al instalar herramientas nuevas, como AIDE o auditd, aparecen comprobaciones nuevas sobre ellas.

Firewall (64 → 66 en Ubuntu, 64 → 65 en Debian). Con UFW activo y la política "denegar todo lo entrante" (la que recomienda el CIS Benchmark), el índice sube 1-2 puntos. En estas VMs no se puede saber qué test exacto sumó esos puntos, porque el log de la pasada inicial se sobrescribió; en Kali, donde se conservan los tres logs, se vio que vienen de los parámetros de kernel que aplica UFW (párrafo siguiente), y es probable que aquí pase lo mismo.

Firewall en Kali (64 → 65). Aquí el caso es distinto. Lynis ya marca el firewall como activo en la pasada inicial (**firewall_software[]=nftables**, **firewall_active=1**), porque Docker carga sus propias reglas en nftables, así que FIRE-4590 da los mismos puntos con y sin UFW. El punto que se gana viene de otro sitio: al activarse, UFW aplica su fichero **/etc/ufw/sysctl.conf**, que desactiva las redirecciones ICMP, y eso resuelve 3 comprobaciones de KRNL-6000 (**accept_redirects** en IPv4 e IPv6). UFW se mantiene igualmente, porque las reglas de Docker solo gestionan el tráfico de los contenedores y no imponen ninguna política sobre lo que entra al equipo.

Medidas que no surtieron efecto. Al revisar los logs del después se ven dos medidas del script que no llegaron a contar. La primera es el banner: Lynis no comprueba solo que **/etc/issue** exista, sino que contenga al menos 5 palabras clave de una lista en inglés (*authorized*, *monitor*, *prohibited*, *legal*…). El texto que pone el script está en español y da 0 coincidencias ("Found only 0 key words" en el log), así que BANN-7126 y BANN-7130 siguen como sugerencia en los tres equipos. La segunda es el **UMASK 027**: el script lo aplica con un **sed** que sustituye la línea **UMASK** de **/etc/login.defs**, pero en Debian 13 y en Kali esa línea no existe, el **sed** no cambia nada y AUTH-9328 sigue como sugerencia; solo se aplicó en Ubuntu. Las dos tienen una corrección sencilla: un banner bilingüe que incluya esas palabras en inglés, y añadir la línea **UMASK 027** cuando no exista. Los índices de la tabla corresponden al script tal como está en el repositorio, sin esta corrección.

Sugerencias de Lynis que no se aplicaron. Después del bastionado quedan entre 35 y 42 sugerencias por equipo. Las principales y el motivo para no aplicar cada una están en el Anexo C. En resumen, se descartaron las que exigen reparticionar o reinstalar (particiones separadas), las que romperían el trabajo del usuario (restringir los compiladores), las que no aplican a un equipo de usuario (SSH) y las que necesitan infraestructura que no existe en este entorno (servidor central de logs).

No se saltó ningún test con el perfil **custom.prf**, para no subir el índice de forma artificial.

Warnings del después. Lynis da un *warning* cuando algo no cumple su política, aunque no sea necesariamente un fallo. En los tres equipos aparece FINT-4316 (AIDE sin base de datos, ver Conclusiones). En Ubuntu aparece además MAIL-8818: el banner SMTP de postfix muestra el nombre del software. Lo provoca el propio bastionado, porque rkhunter y AIDE instalan postfix como dependencia; se corrige ocultando el nombre con la opción **smtpd_banner**, o quitando postfix si el equipo no envía correo. En Debian aparecen PKGS-7392 (paquetes con vulnerabilidades conocidas, que se corrige actualizando con **apt upgrade**) y NETW-2704/2705, que vienen de que la VM tenía configurado un servidor DNS IPv6 que no respondía y no son un problema de seguridad.

Evidencias: capturas de las tres pasadas de cada equipo en las figuras 1 a 9 del anexo, y logs y **report.dat** de Lynis en **evidencias/sistemas/Ubuntu/**, **Debian/** y **Kali/**. En Debian, Lynis muestra además el aviso "Exceptions found" (figuras 4 y 6): según el log, lo genera el test HTTP-6624, que encuentra en la configuración de Apache rutas que no sabe interpretar. Es un aviso de la propia herramienta, no un fallo de seguridad, y aparece igual antes y después del bastionado.

1.3 Conclusiones

Con un único script, idempotente y aplicable sin cambios a las tres distribuciones, los tres equipos pasan de un índice de 64 a 71-73, por encima del 70 exigido. El firewall aporta 1-2 puntos y el resto de grupos 6-8, la mayor parte de ellos por las herramientas de detección y auditoría (rkhunter, fail2ban, AIDE, auditd, acct). A las instalaciones por defecto de Ubuntu, Debian y Kali les falta sobre todo detectar y registrar lo que ocurre en el sistema.

La revisión de los logs mostró además que el banner y el **UMASK** se aplicaron sin errores pero Lynis no los cuenta, por el idioma del banner y porque la línea **UMASK** no existe en Debian ni en Kali. Por eso cada medida debe comprobarse también en el log de Lynis.

Queda pendiente en los tres equipos la configuración de las herramientas instaladas. El script instala AIDE pero no crea su base de datos de referencia (**aideinit**), y sin ella AIDE no puede detectar cambios en los ficheros (Lynis lo avisa con FINT-4316). Lo mismo pasa con auditd, que se instala sin reglas (y en Kali ni siquiera queda activo). Los siguientes pasos serían inicializar AIDE, definir reglas de auditd, corregir el banner y el **UMASK**, y bloquear los protocolos de red que no se usan.

2. Análisis y bastionado de navegador web

2.1 Descripción del sistema analizado

Se bastiona Firefox en la VM **Pai-ubuntu** (Ubuntu 24.04.5, la misma de la Parte 1). Las directivas no se configuran a mano en los ajustes: se aplican como política corporativa, un fichero **policies.json** que Firefox lee al arrancar y que el usuario no puede cambiar. El fichero está en **navegador/firefox/policies.json** y se despliega en **/etc/firefox/policies/** con **sudo navegador/desplegar.sh firefox**.

Medidas aplicadas

Directiva	Valor	Riesgo que mitiga
HttpsOnlyMode	**force_enabled**	Tráfico sin cifrar y ataques de *downgrade* a HTTP
SSLVersionMin	**tls1.2**	Conexiones con TLS 1.0/1.1, obsoletos
PasswordManagerEnabled, **OfferToSaveLogins**	**false**	Robo de las contraseñas guardadas en el navegador
DisableFormHistory	**true**	Datos de formularios guardados en el equipo
DNSOverHTTPS	activado y bloqueado	Espionaje y manipulación de las consultas DNS
EnableTrackingProtection	activado y bloqueado, con bloqueo de criptominado y *fingerprinting*	Rastreo entre webs y huella digital del navegador
Cookies	**reject-tracker-and-partition-foreign**	Cookies de rastreo de terceros
Permissions (cámara, micrófono, ubicación, notificaciones)	**BlockNewRequests**	Webs que piden permisos intrusivos
ExtensionSettings	todo bloqueado salvo uBlock Origin (instalado a la fuerza)	Extensiones maliciosas
BlockAboutConfig	**true**	Que el usuario se salte la política desde **about:config**
DisableTelemetry, **DisableFirefoxStudies**	**true**	Envío de datos de uso a terceros

Problemas encontrados y cómo se solucionaron

- **Políticas inactivas tras el despliegue.** Aunque el fichero estaba en **/etc/firefox/policies/policies.json**, **about:policies** mostraba "The Enterprise Policies service is inactive". La causa eran los permisos: el fichero se copiaba desde la carpeta compartida de VirtualBox, cuyos ficheros no tienen lectura para "otros", y quedaba como propiedad de root sin permiso de lectura para el usuario que ejecuta Firefox. Se corrigió con **chmod 644** sobre el fichero (y **755** sobre la carpeta), y **desplegar.sh** instala ahora el fichero con **install -m 644** para que no vuelva a pasar.
- **Firefox no recargaba la política.** En Ubuntu, Firefox es un *snap* y puede seguir en segundo plano después de cerrar la ventana, sin volver a leer la política. Se cierra del todo con **pkill -f firefox** antes de comprobar

about:policies.

2.2 Plan de pruebas y resultados

Prueba	Web	Antes	Después	Figuras
Política aplicada	**about:policies**	"The Enterprise Policies service is inactive"	Aparecen todas las directivas del **policies.json**	10, 11
DNS cifrado	1.1.1.1/help	DoH: No (resolvía el DNS del operador, Orange)	DoH: **Yes**, conectado a 1.1.1.1 (Cloudflare)	12, 13
Rastreo	Cover Your Tracks (EFF)	"Some protection… some gaps": anuncios y rastreadores invisibles bloqueados solo en parte	"**Strong protection**": anuncios y rastreadores invisibles bloqueados	14, 15
HTTP sin cifrar	http.badssl.com	Carga la página por HTTP, solo con el aviso "Not Secure"	Bloqueada: aviso "HTTPS-Only Mode Alert · Secure Site Not Available"	16, 17
Contraseñas	Login de prueba (the-internet.herokuapp.com)	Firefox ofrece guardar la contraseña	Inicia sesión sin ofrecer guardarla	18, 19

2.3 Conclusiones

Con un único fichero de políticas, desplegado por script y sin tocar los ajustes a mano, mejoran las cinco pruebas: el tráfico sin cifrar se bloquea, el DNS va cifrado, el navegador ya no guarda contraseñas y la protección contra el rastreo pasa de parcial a fuerte. Al ser una política del sistema, el usuario no puede desactivarla, y **about:config** queda bloqueado. Lo único que no cambia es la huella digital: Cover Your Tracks la sigue considerando única, porque la protección estándar de Firefox no la oculta del todo. Ocultarla exigiría **privacy.resistFingerprinting**, que se descarta porque rompe muchas webs. El principal problema del despliegue fueron los permisos del fichero, ya corregidos en el script.

3. Auditoría de sistema móvil

3.1 Descripción del sistema analizado y herramientas de análisis

Se auditaron 2 móviles Android personales (BYOD), ambos Google Pixel: un Pixel 9 Pro y un Pixel 10, los dos con Android 17 y parche de seguridad del 05-10-2026. Se comprobaron los 8 puntos de la política en los dos móviles. Las comprobaciones se hicieron mediante ADB (Android Debug Bridge), con dos scripts propios: **movil/auditar_android.sh**, que vuelca por consola el estado de cada punto comprobable, y **movil/conexiones_b2.sh**, que lista las conexiones abiertas y la app que tiene cada una (punto B2); y Malwarebytes como herramienta de escaneo antimalware, instalada en cada móvil para el punto B1. Se eligió Malwarebytes frente a MobSF o Koodous porque analiza el móvil completo desde el propio dispositivo y sin root; MobSF analiza APKs sueltos y está pensado para auditar una app concreta, no la postura de un dispositivo.

3.2 Plan de pruebas y resultados

Procedimiento en cada móvil: activar Opciones de desarrollador y Depuración USB, volcado **antes** con **auditar_android.sh**, **conexiones_b2.sh** con Chrome en uso, escaneo con Malwarebytes, corrección de lo que falle, volcado **despues**, y desactivar la Depuración USB al terminar. El escaneo de Malwarebytes se hizo una vez por móvil, antes de corregir, porque las correcciones (Bluetooth, redes Wi-Fi, actualizar Chrome y cerrar apps) no instalan ni quitan aplicaciones y no cambian el resultado de B1. La única app nueva entre los dos volcados del Pixel 9 Pro es el propio Malwarebytes.

Resultados (detalle completo en **movil/checklist.md**):

Punto	Pixel 9 Pro	Pixel 10
A1. Orígenes desconocidos desactivados	Cumple	Cumple
A2. Sin root	Cumple	Cumple
A3. Sistema y navegador al día	Cumple	**No cumplía** (Chrome)
A4. Sin Wi-Fi inseguras	Cumple	**No cumplía**
A5. Bluetooth apagado	**No cumplía**	Cumple
B1. Sin malware	Cumple	Cumple
B2. Sin conexiones de fondo no autorizadas	**No cumplía**	**No cumplía**
B3. Sin permisos peligrosos	Cumple	**No cumplía**

El punto A1 se comprobó a mano en cada móvil (Ajustes → Instalar aplicaciones desconocidas: ninguna app con el permiso concedido) y con el instalador de cada app de terceros que da el script: todas vienen de Play Store salvo, en el Pixel 9 Pro, una WebAPK instalada por Chrome (**org.chromium.webapk...**), que es una web añadida a la pantalla de inicio y no un APK de origen desconocido. El A3 se comprobó en Ajustes → Actualizaciones de software y en la ficha de Chrome en Play Store, porque el script solo da el nivel de parche. En B3 el script lista las apps con servicios de accesibilidad (ninguna en los dos móviles), lectores de notificaciones y permiso de superposición; en estos dos últimos solo aparecen componentes del sistema y de Google (lanzador, Android Auto, búsqueda, teléfono y cámara), ninguna app de terceros. El acceso al almacenamiento compartido, que la política también incluye en B3, se comprobó con **dumpsys package**, que indica si el permiso está concedido de verdad (la lista de **appops** incluía apps a las que Android ya se lo había retirado). Ninguna app tiene acceso total al almacenamiento en ninguno de los dos móviles, y en el Pixel 9 Pro ninguna de sus 81 apps de terceros puede leer el almacenamiento compartido. En el Pixel 10, en cambio, 6 apps de terceros podían leer el almacenamiento compartido, y con él los ficheros que descarga el navegador: un explorador de archivos, Adobe Reader, WhatsApp, Dropbox, Telegram y Séneca. Se les retiró el permiso con **adb shell pm revoke**; si alguna lo necesita, lo volverá a pedir al usuario.

Los dos móviles fallaron en el B2, que se explica más abajo. Aparte, en el Pixel 9 Pro falló el Bluetooth encendido, corregido apagándolo, y en el Pixel 10 hubo tres fallos más: A3 y A4, que se explican a continuación, y B3, que se explica junto con la comprobación de permisos. A3: el sistema estaba al día, pero Chrome tenía una actualización pendiente en Play Store, y se corrigió actualizándolo (la ficha pasa de "Actualizar" a "Abrir"). A4: tenía 6 redes Wi-Fi guardadas sin cifrado (**Andared_IoT**, **Carrefour-Conecta**, **QUIRONSALUD**, **Quironsalud_Invitados**, **VuelingWi-Fi** y una variante abierta de **paradores**), típicas de wifis públicas de aeropuertos, hospitales o comercios; se corrigió olvidando esas 6 redes, verificado con **adb shell cmd wifi list-networks** antes y después. Además tenía 2 redes guardadas con SSID oculto y clave compartida, sin autenticación Enterprise (**Livebox7-789E-WiFi7** y **alternative-wifi-etsi**), que la política también prohíbe. Se detectaron con **adb shell dumpsys wifi**, porque **list-networks** no indica si una red está oculta, y se corrigieron olvidándolas. En el Pixel 9 Pro ninguna de las redes guardadas está oculta.

Punto B2. En cada móvil, con Chrome abierto y navegando, **conexiones_b2.sh** lee las tablas de conexiones del kernel (**/proc/net/tcp**, **tcp6**, **udp** y **udp6**), que incluyen el UID de cada conexión, y las cruza con **pm list packages -U** para saber qué app tiene cada una. Se incluye UDP porque Chrome y otras apps de Google usan QUIC, que va sobre UDP; con solo TCP no habría aparecido ninguna app de terceros. Se consideran autorizados el navegador y los componentes preinstalados del sistema (servicios de Google Play, Play Store, pila de red, búsqueda y widget del tiempo). En los dos había apps que el usuario había abierto antes y seguían conectadas en segundo plano. En el Pixel 9 Pro, la primera pasada dio 42 conexiones de 8 apps, entre ellas dos de terceros (**com.chess** y **com.anthropic.claude**) y YouTube. En el Pixel 10 dio 115 conexiones de 7 apps, entre ellas dos de terceros, las apps de los periódicos La Razón (**es.larazon**, con 74 conexiones) y Marca (**com.iphonedroid.marca**), además de Google TV. Se corrigió forzando la detención de esas apps (**adb shell am force-stop**). En la segunda pasada, también con Chrome en uso, quedan 26 conexiones en el Pixel 9 Pro y 44 en el Pixel 10, todas de Chrome o de componentes del sistema.

Evidencia: Chrome desactualizado en el Pixel 10 (figura 20) y escaneos de Malwarebytes sin amenazas (figuras 21 y 22); logs de ADB en **evidencias/movil/<modelo>-antes/** y **evidencias/movil/<modelo>-despues/** (**auditoria.txt**, **conexiones-b2.txt** y, para las redes ocultas y el almacenamiento, **a4-b3-extra.txt**), y capturas del

permiso de instalar apps desconocidas (A1) y de las actualizaciones de sistema y de Chrome (A3) en las carpetas **-antes**.

3.3 Conclusiones

Los dos móviles partían de un nivel de seguridad razonable, sin root, sin apps de origen desconocido y sin malware, pero los dos tenían fallos de higiene: los dos con apps que el usuario creía cerradas y seguían conectadas en segundo plano, el Pixel 9 Pro además con el Bluetooth encendido sin necesidad, y el Pixel 10 con el navegador sin actualizar y arrastrando redes Wi-Fi abiertas de sitios públicos y dos redes ocultas sin Enterprise, además de 6 apps de terceros con permiso para leer las descargas del navegador. Ninguno es grave por sí solo, pero todos amplían la superficie de ataque: con el Bluetooth activo hay más vías de conexión no solicitada, una app que sigue conectada en segundo plano mientras se usa el navegador corporativo podría enviar datos sin que el usuario lo sepa, un navegador desactualizado puede tener vulnerabilidades ya públicas, y una red abierta guardada permite que el móvil se reconecte solo a una red que puede estar controlada por un atacante. Son fallos que se acumulan con el uso diario sin que el usuario lo note, así que conviene revisarlos de forma periódica y no solo una vez.

4. Auditoría de sistema web

4.1 Descripción del sistema web y herramientas de análisis

El objetivo es SOS-hardened, un fork del proyecto SOS2526-11 (asignatura SOS, curso anterior), del que uno de los integrantes de este equipo es coautor. Se audita en local, nunca contra la URL real que el equipo de SOS tiene desplegada en Render, por tratarse de un servicio compartido con terceros ajenos a esta práctica.

El enunciado permite auditar "aplicaciones en desarrollo propias", además de los entornos de pruebas que propone (DVWA, OWASP Juice Shop, WebGoat…). Se eligió SOS-hardened porque es una aplicación propia que no está en producción, y porque sus fallos son reales y no se han puesto a propósito, como pasa en esos laboratorios.

Stack: backend Node.js/Express con base de datos NeDB (ficheros planos) y autenticación JWT; frontend SvelteKit. Herramientas: OWASP ZAP 2.17 como proxy de interceptación y para las pruebas (Requester y Fuzzer), y **hashcat** (modo 16500, JWT) para comprobar la fortaleza del secreto de firma.

Las vulnerabilidades se clasifican según OWASP Top 10:2021, la edición con la que ZAP 2.17 etiqueta sus alertas. En la edición 2025 cambian de número: A02:2021 pasa a A04 (fallos criptográficos), A03:2021 a A05 (inyección), A05:2021 a A02 (configuración incorrecta) y A06:2021 a A03 (cadena de suministro de software); A01 y A07 mantienen su número.

4.2 Plan de pruebas y resultados

Trazabilidad del tráfico. Con ZAP escuchando en **localhost:8082** y Firefox configurado a mano para usar ese proxy también en HTTPS (capturas **#5-zap-proxy-8082.png** y **#5-firefox-proxy-manual.png**) se navegó por la aplicación sin modificar ninguna petición. La pestaña History de ZAP registra cada petición y respuesta, incluida la del login, con sus cabeceras y su cuerpo, lo que demuestra que todo el tráfico HTTP pasa por el proxy sin alterarse (figura 28).

Para demostrar también la trazabilidad en HTTPS, se importó el certificado raíz de ZAP en Firefox (Autoridades → Importar → "Confiar en esta CA para identificar sitios web") y se navegó con el proxy activo a **https://example.com**, un dominio externo ajeno a esta práctica usado únicamente para comprobar la intercepción. El certificado de esa página, visto desde el propio navegador, tiene como emisor "Zed Attack Proxy Root CA" en vez del emisor real del sitio, lo que confirma que ZAP genera un certificado propio por cada dominio y que el tráfico cifrado pasa igual por el proxy; la petición **https://example.com/** con **200 OK** queda también registrada en la pestaña History. El certificado con emisor ZAP se ve en la figura 23 y la petición HTTPS en History en la figura 29; la captura **#5-cert-zap-importado.png** (CA de ZAP en Autoridades) está en **evidencias/web/capturas/**.

Metodología seguida, en caja negra: reconocimiento navegando la aplicación a través del proxy, para que ZAP registrara sus rutas (incluida la API, versionada en **v1** y **v2**, con tres recursos cada una); prueba de autenticación en cada endpoint de escritura, sin ninguna cabecera; localización del endpoint real de login observando el tráfico generado al enviar el formulario; y, una vez obtenido un token, análisis de su cabecera y crackeo del secreto de firma con una wordlist construida a partir de los nombres del equipo visibles en la propia web.

Vulnerabilidad 1. Control de acceso roto (A01:2021). Se probó **DELETE** sin autenticación contra los 6 endpoints de recurso de la API (3 recursos × 2 versiones):

Recurso	v1	v2
alcohol-consumptions-per-capita	sin protección	protegido
literacy-rates	sin protección	sin protección
road-fatalities	sin protección	sin protección

5 de 6 aceptan el borrado completo de la colección sin pedir ningún tipo de credencial (figura 24).

Vulnerabilidad 2. Secreto de firma JWT débil y recuperable (A02:2021). El único endpoint protegido (alcohol v2) usa un token JWT firmado con HS256. Tras obtener un token válido, se guardó y se atacó offline con **hashcat**: el diccionario genérico (**rockyou.txt**) no lo recupera, pero una wordlist de 360 combinaciones (palabras típicas de contraseña + nombres del equipo, sacados de la propia página de inicio) lo crackea en menos de un segundo (figura 25). Con el secreto en mano se puede firmar un token válido para cualquier usuario sin pasar por el login. Aparte, el secreto sigue recuperable en el historial de git aunque se rote en el código actual (repo público); rotarlo en el HEAD no deshace la exposición pasada, por eso se trata como comprometido para siempre y no se intenta "arreglar" reescribiendo el historial.

Vulnerabilidad 3. Credenciales por defecto (A07:2021). El login de ese mismo endpoint acepta **admin**/**admin** (figura 26).

Vulnerabilidad 4. Inyección NoSQL (A03:2021), adicional. Los filtros de los **GET** de la API pasan los parámetros de la query string directamente a la base de datos sin comprobar su tipo. Fuzzing con ZAP sobre el parámetro **nation** con 5 operadores de NeDB (**$ne**, **$gt**, **$regex**, **$exists**, **$where**; lista en **web/payloads/nosql.txt** y captura **#4-fuzzer-payloads-nosql.png**) muestra que los 5 se saltan el filtro por igual (figura 27): la petición de control devuelve 342 bytes (3 resultados), y las 5 inyectadas devuelven 17.148 bytes, el tamaño exacto de la colección completa. El impacto en este endpoint concreto es bajo, porque es público y sin filtro ya devuelve todo; el riesgo está en que el mismo patrón de código, sin saneamiento de tipos, se repite en otros puntos de la aplicación, y que el operador **$where** podría usarse para ejecutar código si se le pasara una función en vez del valor simple usado en la prueba.

Dependencias desactualizadas (A06:2021), adicional. npm audit reporta 35 vulnerabilidades (11 moderate, 19 high, 5 critical). La más relevante es **nedb**, la propia base de datos de la aplicación en producción: tiene una vulnerabilidad crítica de Prototype Pollution sin parche disponible. El resto de las críticas y altas vienen de dependencias de testing (**newman**, **postman-***), no de producción.

Cabeceras de seguridad (A05:2021), adicional. El escaneo pasivo de ZAP sobre el tráfico capturado da 5 alertas de configuración, repetidas en las respuestas de la aplicación. Faltan la cabecera **Content-Security-Policy** y la protección anti-clickjacking (**X-Frame-Options**), falta **X-Content-Type-Options: nosniff**, el servidor anuncia su tecnología con **X-Powered-By: Express**, y la API responde con **Access-Control-Allow-Origin: ***, porque el backend usa **cors()** sin opciones (las dos cabeceras se ven en la respuesta de la figura 24). Ninguna es explotable por sí sola, pero quitan capas de defensa. Sin CSP, nada frenaría un script inyectado que llegara a ejecutarse, y la aplicación guarda HTML sin sanear (ver más abajo). Sin protección anti-clickjacking, otra web puede cargar la aplicación dentro de un iframe. **X-Powered-By** le ahorra al atacante identificar el framework. El CORS abierto permite que cualquier web lea las respuestas de la API desde el navegador de quien la visite, aunque aquí el impacto es bajo, porque los datos de lectura son públicos y el token no va en una cookie.

SQL Injection, XSS y Path Traversal (vectores que cita el enunciado). Se probaron también estos tres vectores, con peticiones manuales con **curl**, aunque la aplicación no usa SQL (NeDB no es relacional, por lo que la inyección equivalente es la NoSQL ya descrita en la vulnerabilidad 4):

- *Path Traversal*: 8 variantes de escape de directorio (**../../..**, codificación simple y doble, **....//**) contra la carpeta estática y contra rutas de la API. Las 8 devuelven **404**. No vulnerable: SvelteKit sirve los estáticos con su propio manejador, sin exponer el sistema de ficheros, y las rutas de la API no toman ningún parámetro de ruta o nombre de fichero.
- *XSS reflejado*: el payload **<script>alert(1)</script>** en parámetros de consulta y en la URL no aparece en ninguna respuesta. No hay eco de parámetros en HTML, así que no es vulnerable por esta vía.
- *XSS almacenado*: un registro con **nation = "<img src=x onerror=alert(document.domain)>"** se guarda y se devuelve igual por la API: el valor es un string, así que pasa el control de tipo, pero nadie valida ni limpia su contenido. El frontend lo muestra en tablas con Svelte, que escapa el texto por defecto, y en gráficos con Highcharts, que filtra etiquetas y atributos peligrosos; no se llegó a confirmar la ejecución del script en el navegador, así que se reporta como entrada sin sanear y no como XSS confirmado. La prueba se hizo con un token válido sobre la versión ya corregida, por lo que este punto sigue abierto después de la mitigación (ver 4.4).

Evidencia de los tres en **evidencias/web/xss-path-traversal-2026-11-08.log**.

CSRF. No aplica. El frontend guarda el token en **sessionStorage** y lo envía en la cabecera **Authorization**, que el navegador no añade por su cuenta a las peticiones que lanza otra web. Sin una cookie de sesión, una web ajena no puede hacer peticiones autenticadas en nombre del usuario.

Evidencia (peticiones, respuestas, resultados de hashcat y log de **npm audit**) en **evidencias/web/capturas/** y **evidencias/web/npm-audit-2026-11-06.log**; informe HTML generado por ZAP en **evidencias/web/2026-11-06-ZAP-Report-localhost.html** y exportación del tráfico capturado en **evidencias/web/zap-history-export.csv**; notas detalladas en **evidencias/web/notas-sesion.md**.

4.3 Plan de mitigación y demostración (antes/después)

Se corrigieron las 4 vulnerabilidades en el código de **SOS-hardened**: un middleware de autenticación (**verifyToken**) centralizado y aplicado a todos los endpoints de escritura de los 6 ficheros de la API (antes solo protegía uno de seis); el secreto de firma JWT y las credenciales de login movidos a variables de entorno, generadas aleatoriamente, fuera del código fuente y del repositorio; y un saneador (**stringFilter**) que descarta cualquier valor de filtro que no sea un string plano, bloqueando la inyección de operadores NoSQL. El cambio completo está en **web/mitigacion-sos-hardened.patch** (9 ficheros, 118 líneas añadidas y 66 eliminadas) y publicado en https://github.com/Ridawo/SOS-hardened.

Repitiendo exactamente los mismos ataques de la sección 4.2:

Vulnerabilidad	Antes	Después
Control de acceso roto	5/6 endpoints en **200 OK** sin credenciales	6/6 en **403 Forbidden**
Secreto JWT débil	Token forjado con el secreto expuesto, aceptado	Rechazado (**401**); secreto rotado y fuera del código
Credenciales por defecto	**admin**/**admin** funcionaba	Rechazado (**401**); credenciales aleatorias en **.env**
Inyección NoSQL	El operador **$ne** alteraba el resultado de la consulta	El operador se descarta antes de llegar a la base de datos (verificado comparando **nation[$ne]=Spain** contra la consulta sin filtro: ambas devuelven el mismo número de resultados, 150, cuando una inyección real habría dado 147)

Se comprobó también que la funcionalidad legítima sigue intacta: el login con las credenciales nuevas devuelve un token válido, y ese token permite realizar un **PUT** real sobre un recurso.

Para lo que no se corrigió en el código, el plan de mitigación recomienda estos controles:

Hallazgo	Control recomendado
Cabeceras de seguridad (A05)	Añadir el middleware **helmet** en Express, que pone CSP, **X-Frame-Options** y **X-Content-Type-Options** y quita **X-Powered-By**; limitar **cors()** al dominio del propio frontend
Entrada sin sanear (XSS almacenado)	Validar el contenido de los campos de texto en el servidor (por ejemplo, permitir solo letras, espacios y guiones en **nation**) y mantener el escapado del frontend
Dependencias vulnerables (A06)	Ejecutar **npm audit fix**, sacar las dependencias de testing del build de producción y migrar de **nedb**, que no publica versiones desde 2016 (1.8.0), a una alternativa mantenida como **@seald-io/nedb**
Secreto expuesto en el historial de git	Tratarlo como comprometido: rotarlo (ya hecho) y no reutilizarlo en ningún otro despliegue

4.4 Conclusiones

El sistema no tenía, en la práctica, control de acceso: la autenticación faltaba en la mayoría de los endpoints de escritura, y donde existía dependía de un secreto adivinable con un diccionario pequeño y de credenciales por defecto nunca cambiadas. Eran fallos de diseño, así que la corrección centraliza la autenticación y el saneamiento de entrada en toda la API. Quedan como deuda técnica no resuelta en esta práctica la actualización de dependencias (**npm audit**), en particular **nedb**, que no tiene parche disponible para su vulnerabilidad crítica; la validación del contenido de los campos de texto en los **POST**/**PUT**, que hoy aceptan HTML y solo están protegidos por el escapado del frontend; y las cabeceras de seguridad. Para las tres hay un control concreto en el plan de mitigación (4.3).

Referencias

- OWASP. *OWASP Top 10:2021*. https://owasp.org/Top10/
- OWASP. *OWASP Top 10:2025*. https://owasp.org/Top10/2025/
- CIS (Center for Internet Security). *CIS Benchmarks*. https://www.cisecurity.org/cis-benchmarks
- NIST. *SP 800-115: Technical Guide to Information Security Testing and Assessment*. https://csrc.nist.gov/pubs/sp/800/115/final
- España. Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad (ENS), categoría Alta. CCN-CERT. https://www.ccn-cert.cni.es/
- CISOfy. *Lynis*, herramienta de auditoría de seguridad de sistemas. https://cisofy.com/lynis/
- OWASP. *OWASP ZAP*, versión 2.17. https://www.zaproxy.org/
- Mozilla. *Policy Templates for Firefox*. https://mozilla.github.io/policy-templates/
- Android Developers. *Android Debug Bridge (adb)*. https://developer.android.com/tools/adb
- Malwarebytes. *Malwarebytes para Android*. https://www.malwarebytes.com/android
- *hashcat*, modo 16500 (JWT). https://hashcat.net/hashcat/

Anexos

Anexo A. Capturas

A.1 Sistemas (Lynis)

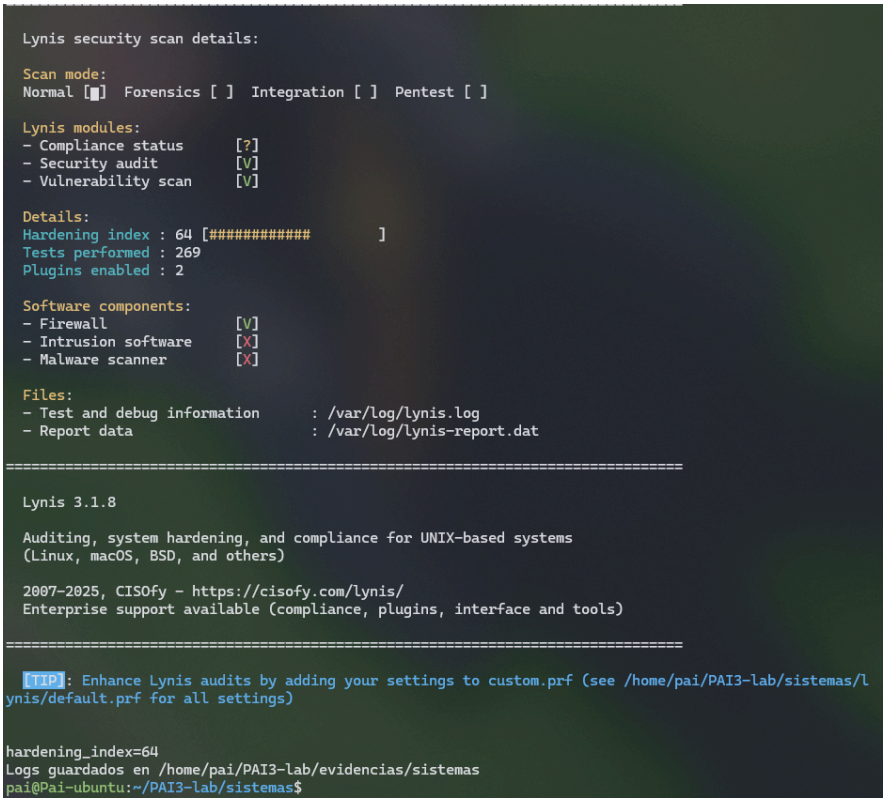


Figura 1: Ubuntu: índice inicial, 64.

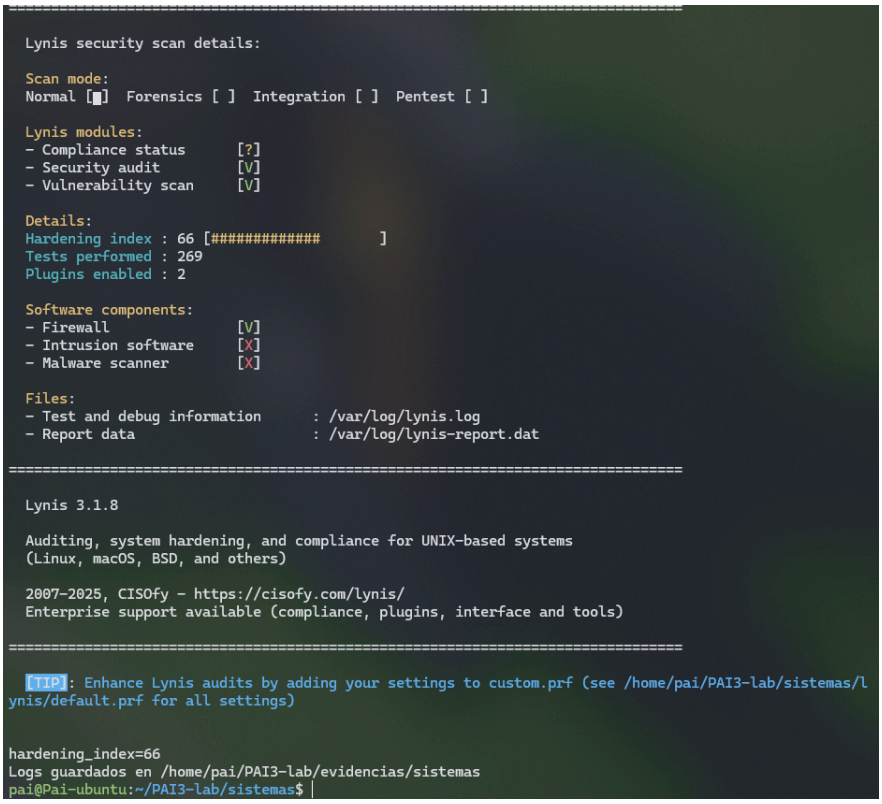


Figura 2: Ubuntu: índice tras activar el firewall, 66.

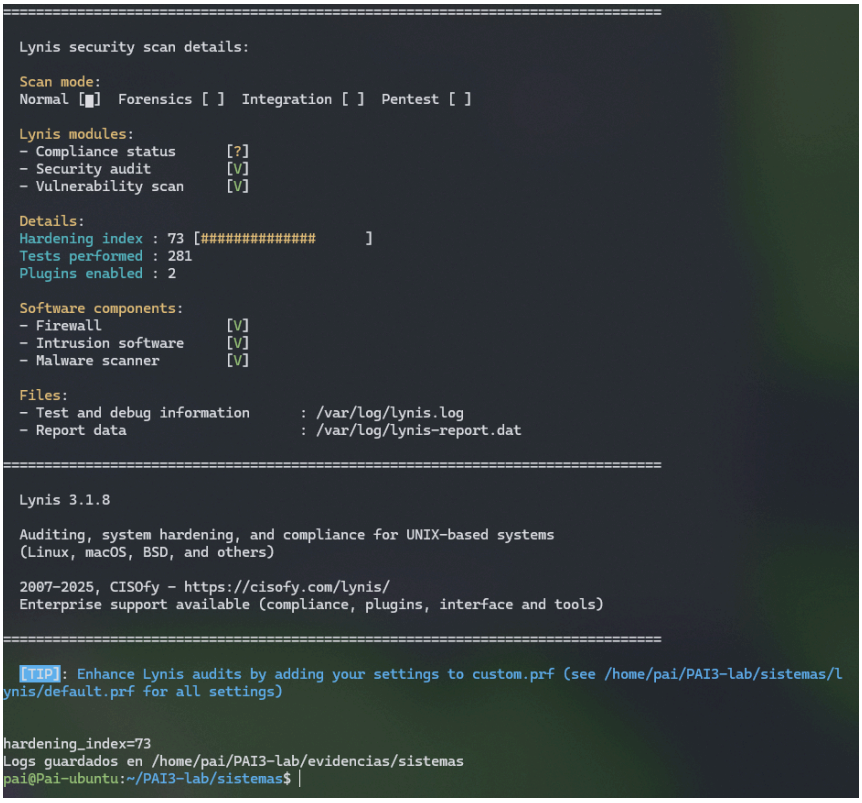


Figura 3: Ubuntu: índice tras el bastionado completo, 73.



Figura 4: Debian: índice inicial, 64.

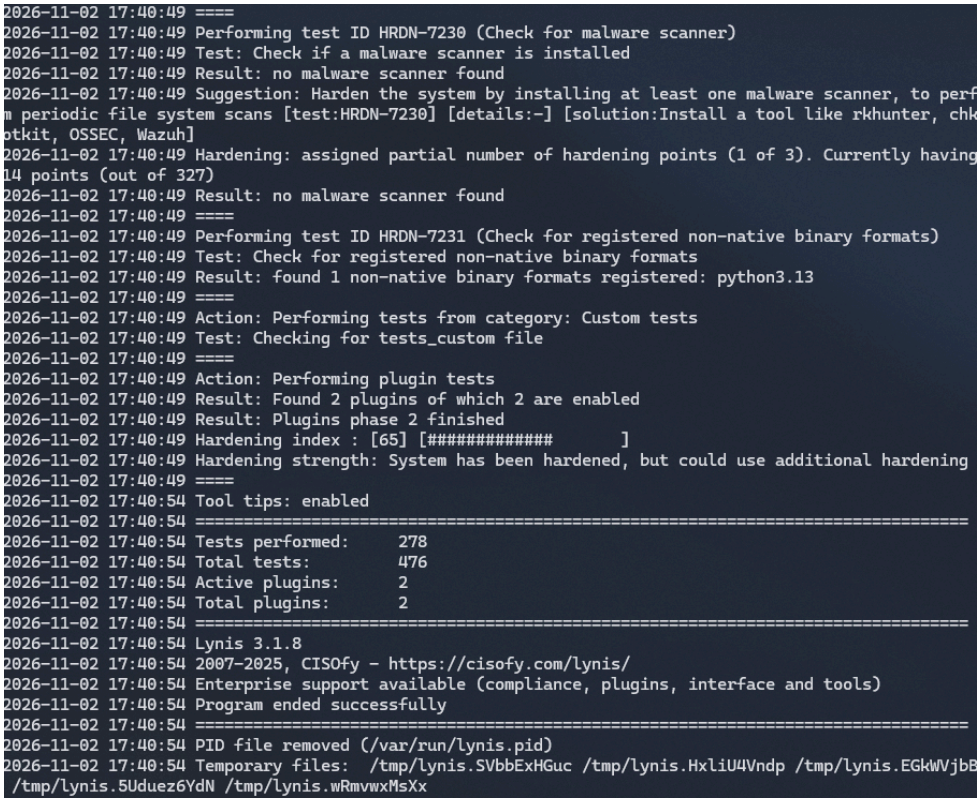


Figura 5: Debian: índice tras activar el firewall, 65 (extracto de **lynis.log**).

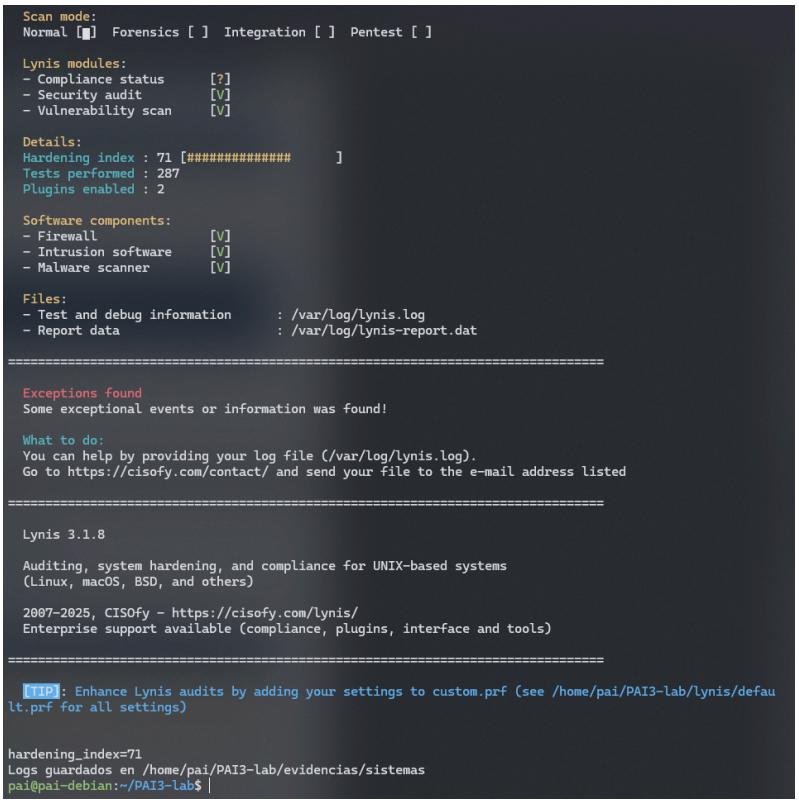


Figura 6: Debian: índice tras el bastionado completo, 71.

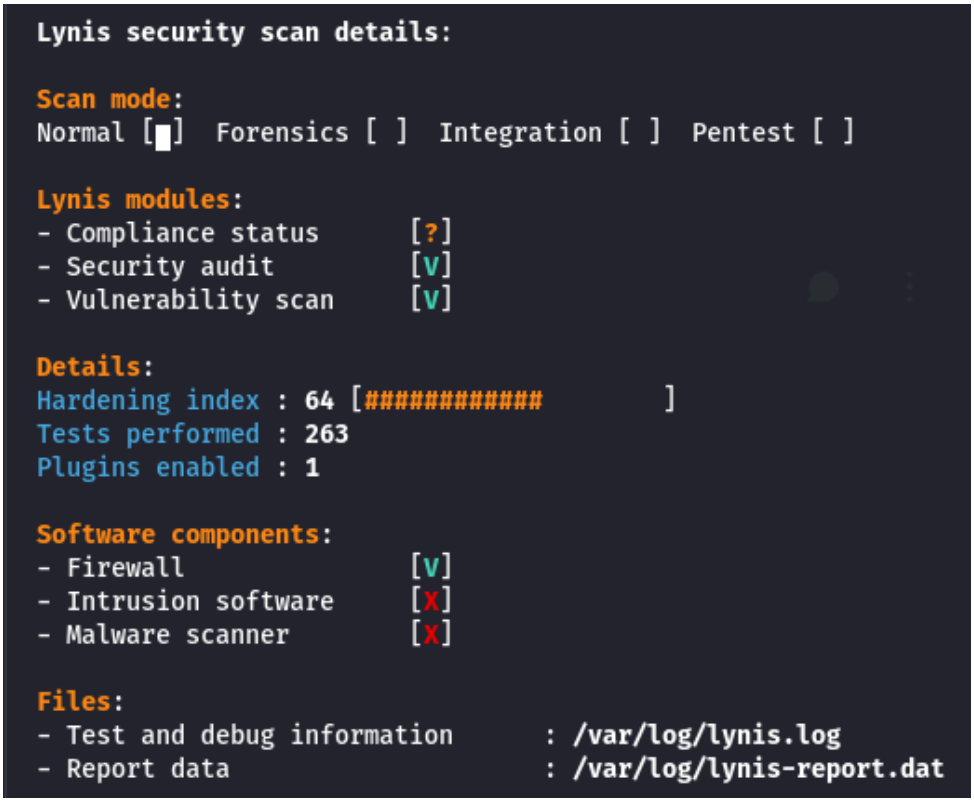


Figura 7: Kali: índice inicial, 64.

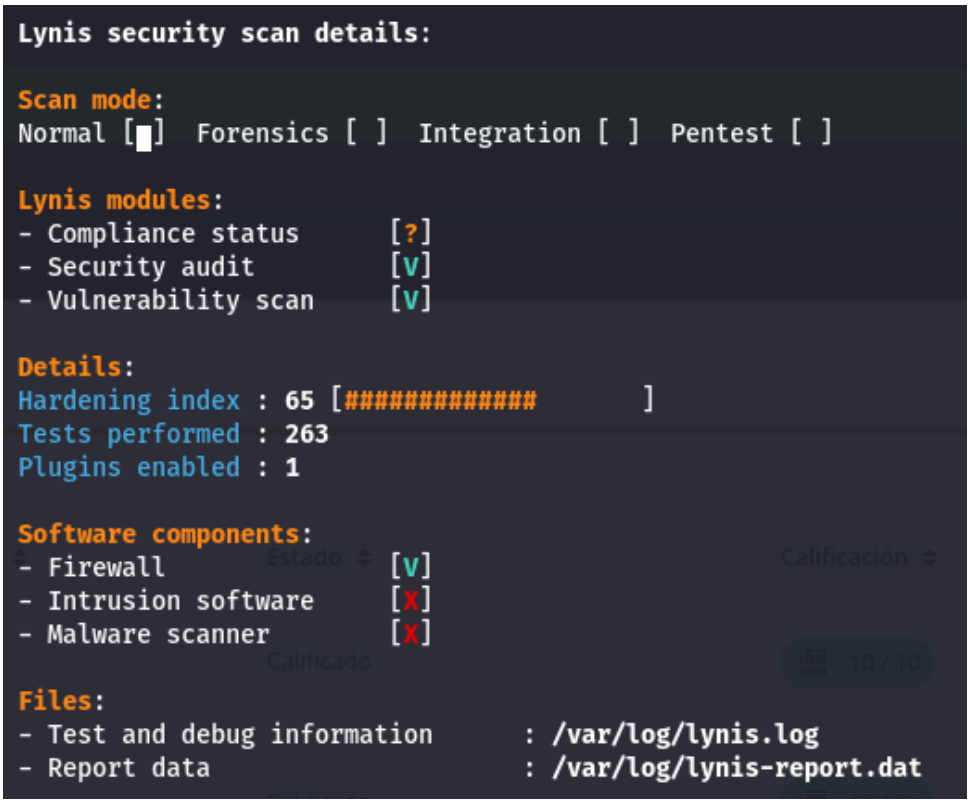


Figura 8: Kali: índice tras activar el firewall, 65.

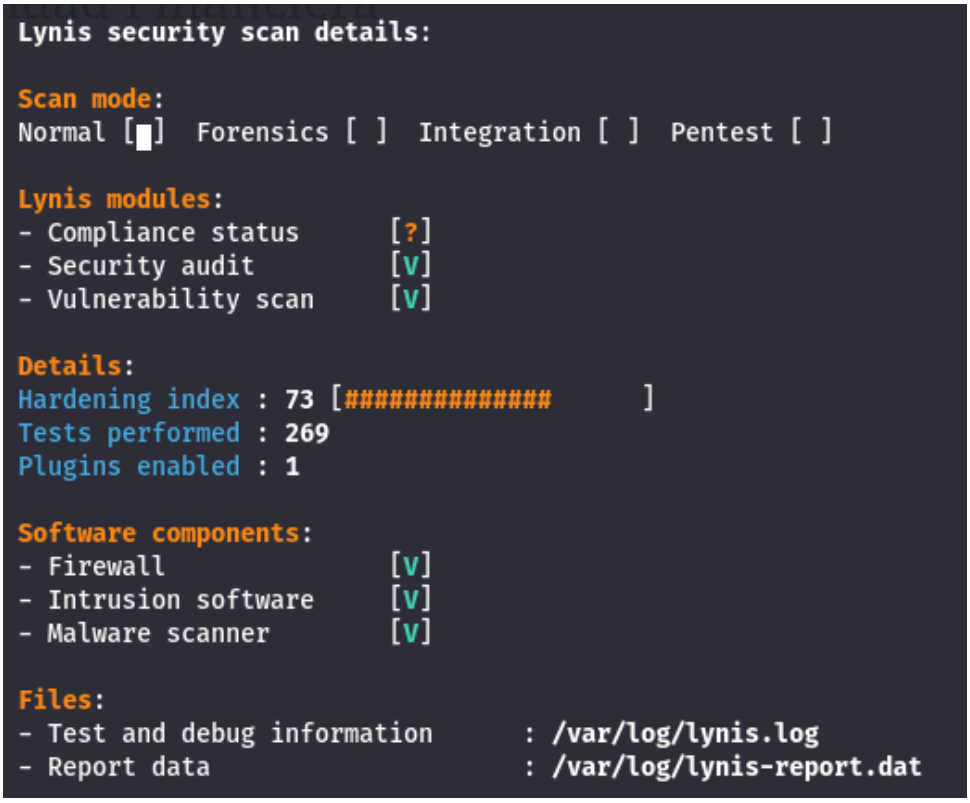


Figura 9: Kali: índice tras el bastionado completo, 73.

A.2 Navegador (Firefox)

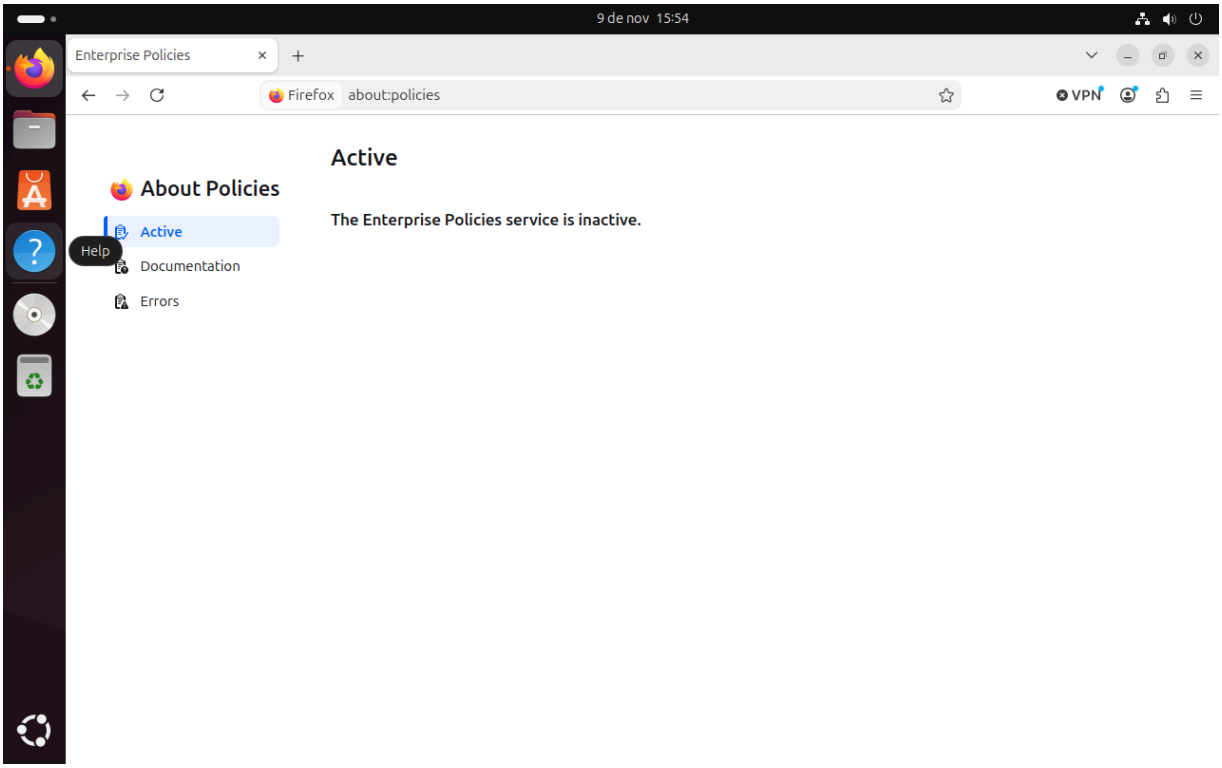


Figura 10: **about:policies** antes de corregir los permisos: servicio de políticas inactivo.

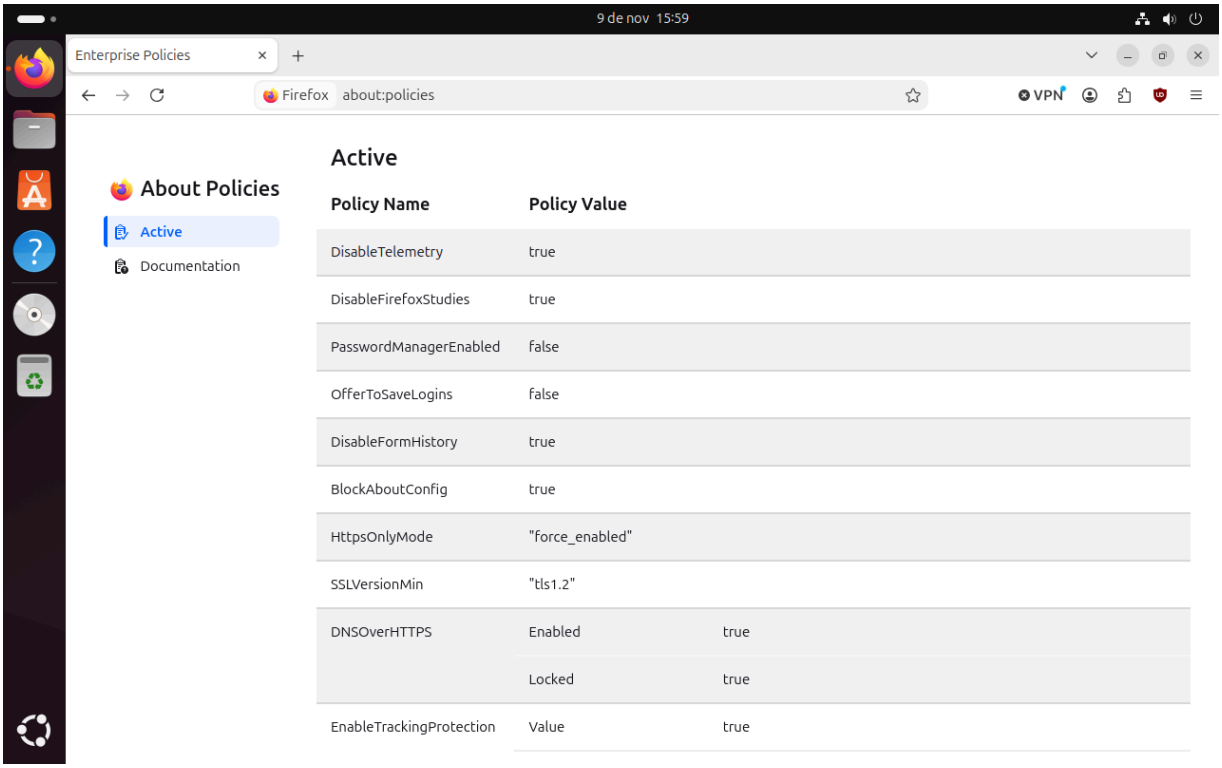


Figura 11: **about:policies** después: directivas activas.

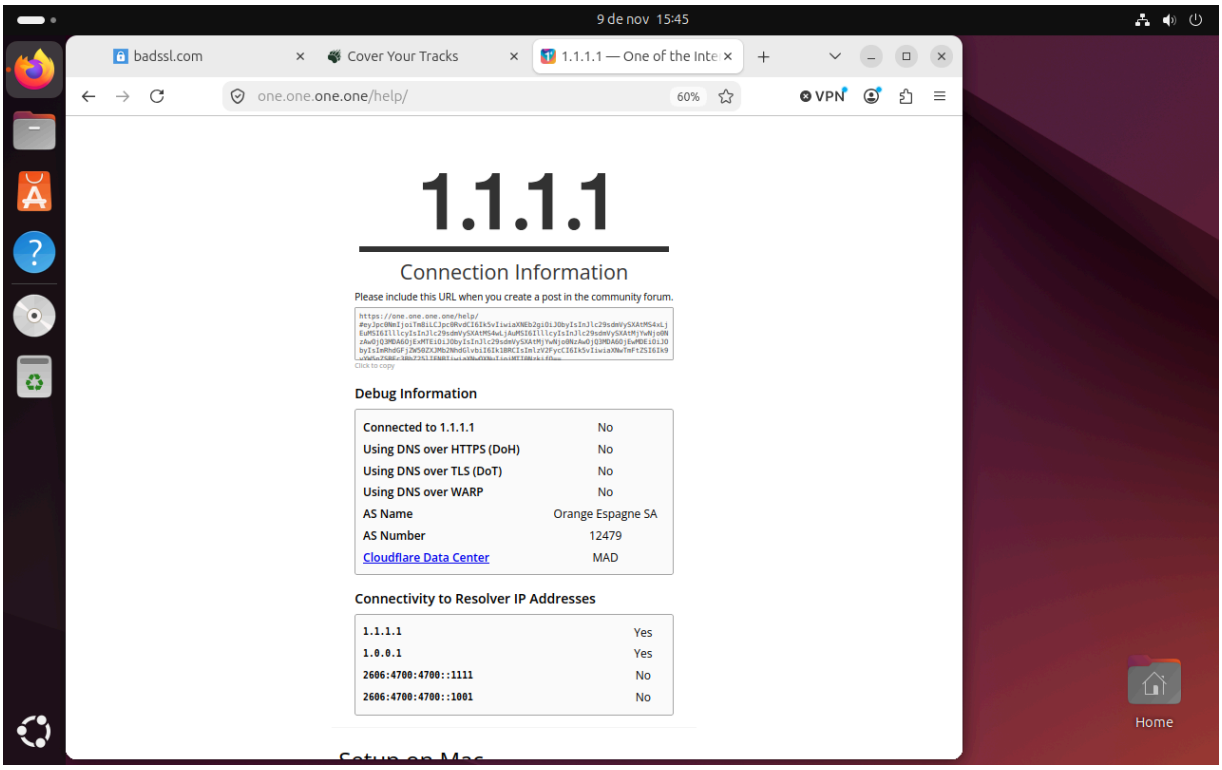


Figura 12: 1.1.1.1/help antes: sin DNS over HTTPS.

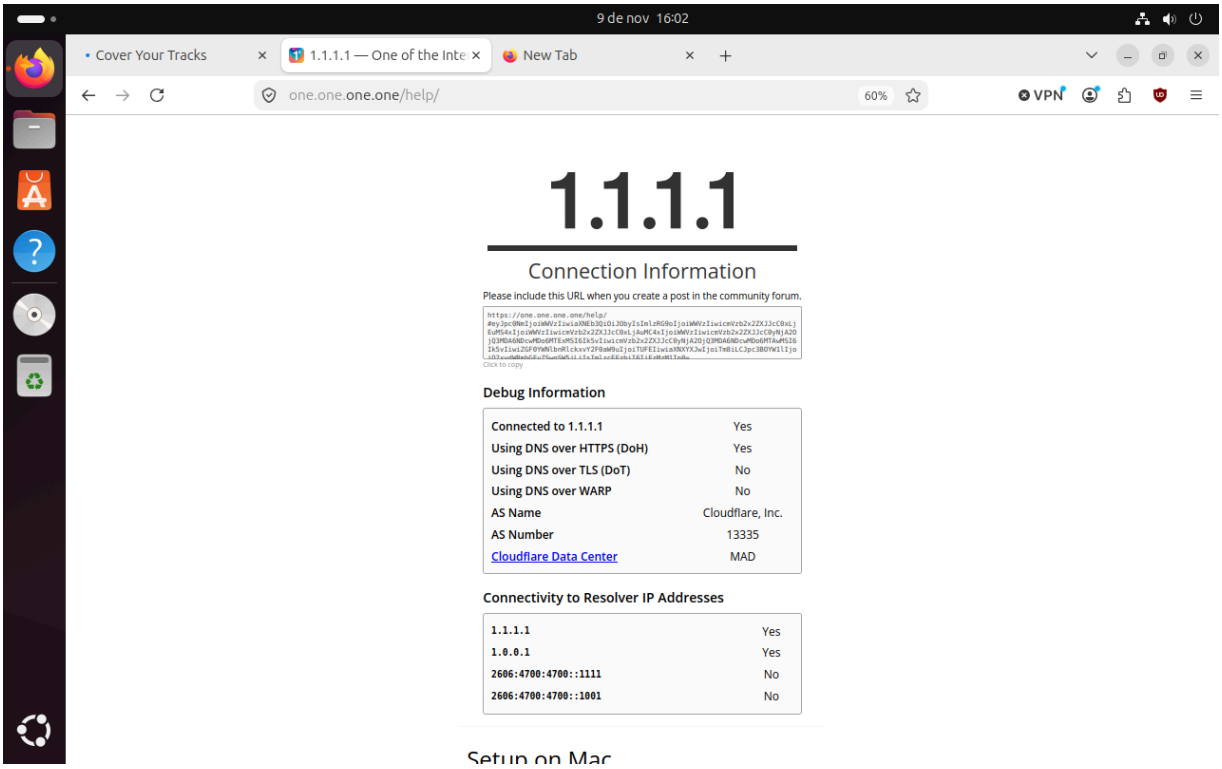


Figura 13: 1.1.1.1/help después: DNS over HTTPS activo.

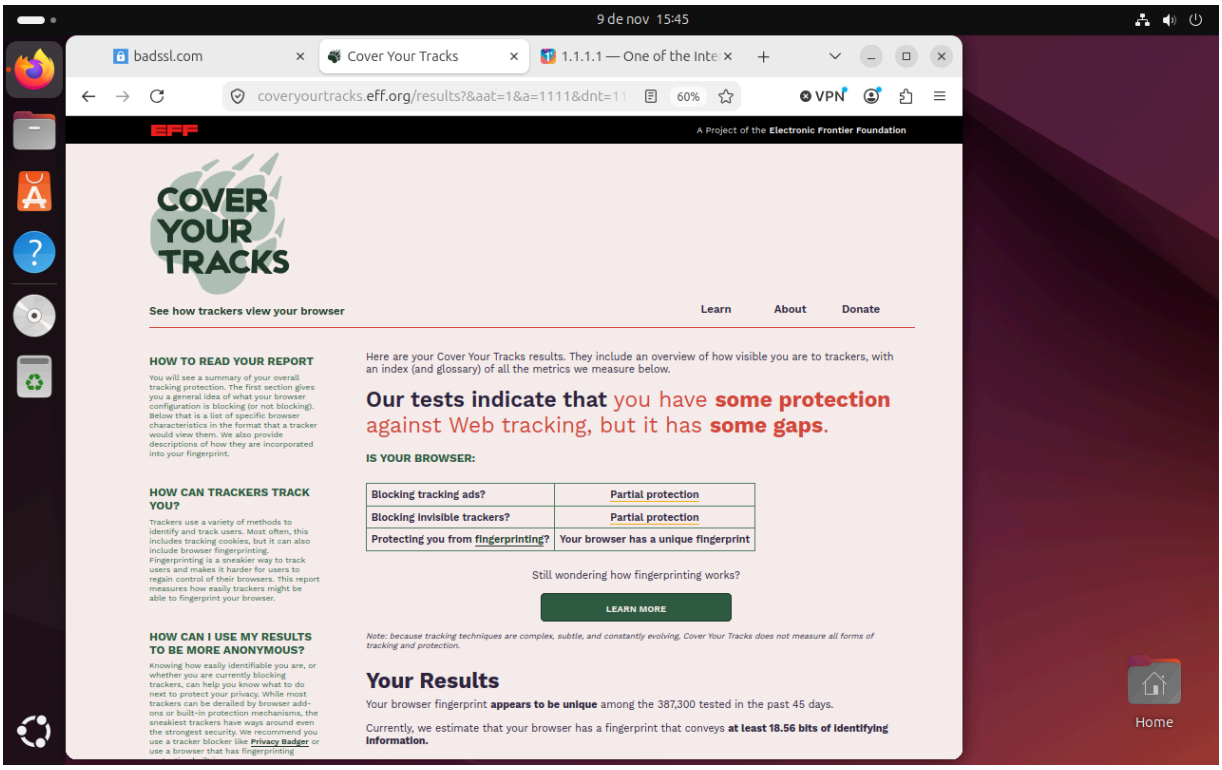


Figura 14: Cover Your Tracks antes: protección parcial.

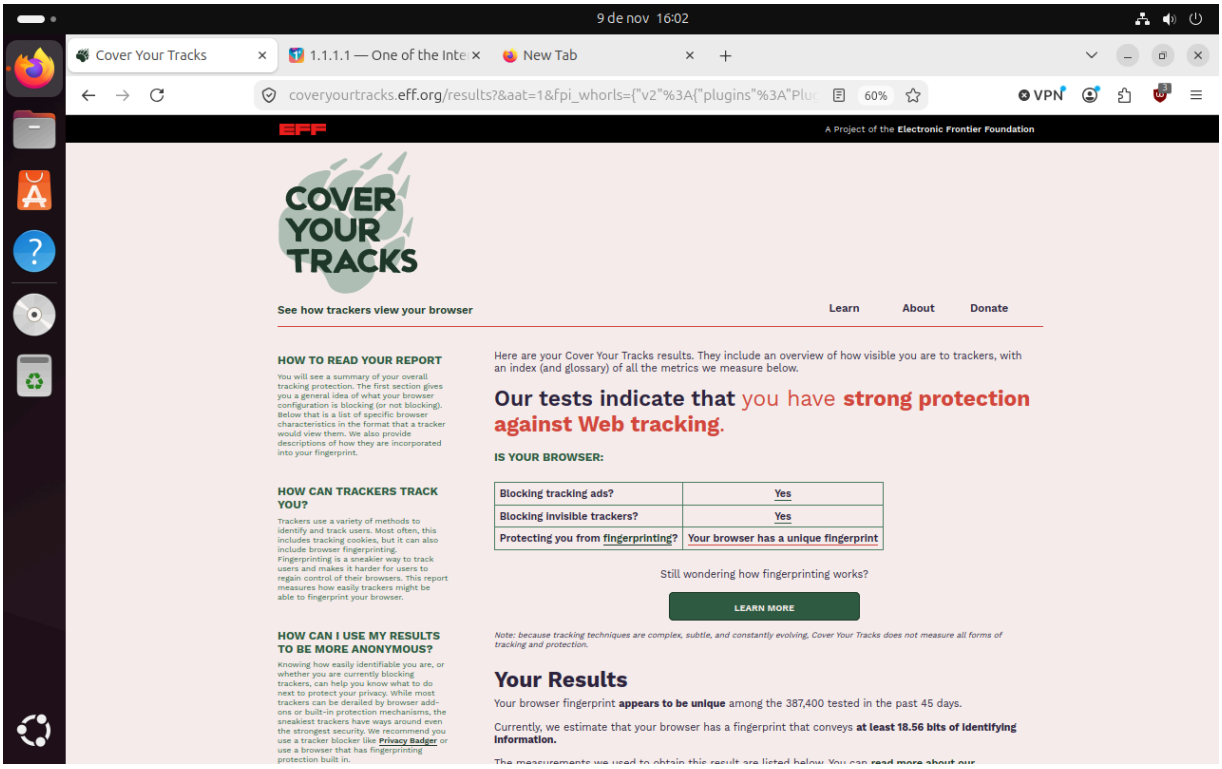


Figura 15: Cover Your Tracks después: protección fuerte.

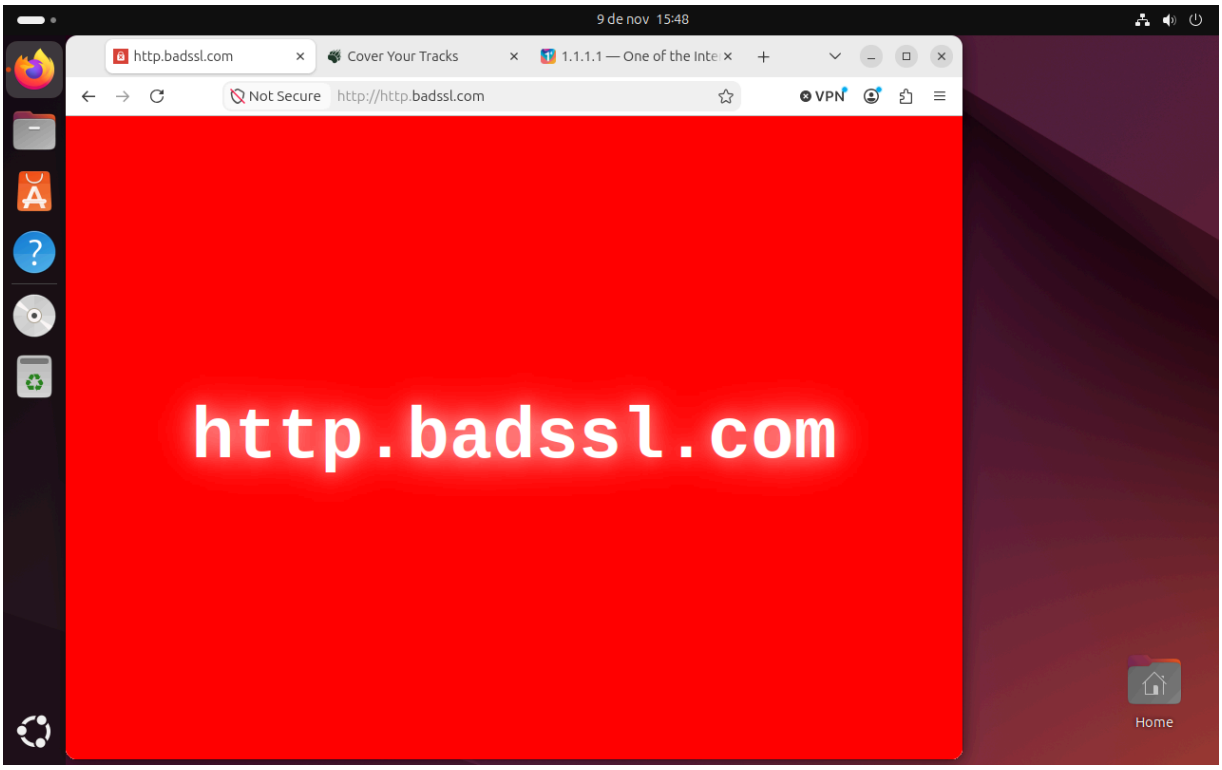


Figura 16: http.badssl.com antes: carga por HTTP sin cifrar.

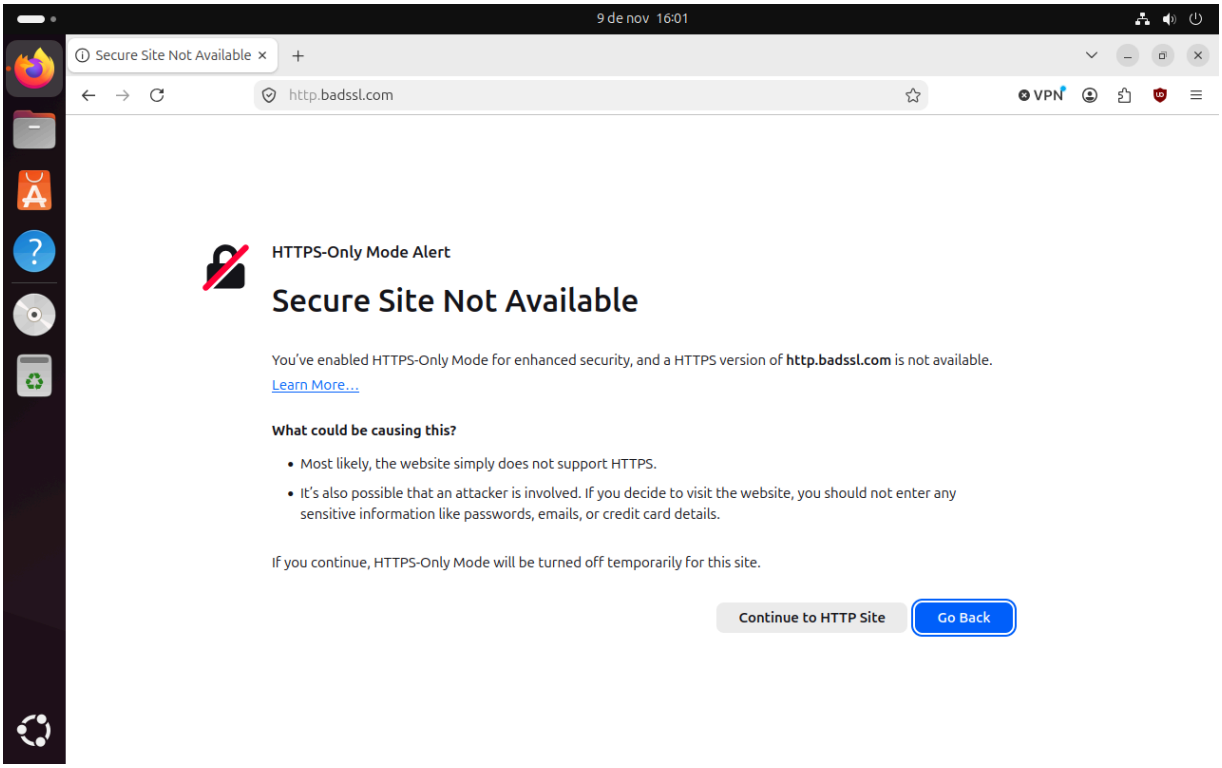


Figura 17: http.badssl.com después: bloqueada por el modo solo HTTPS.

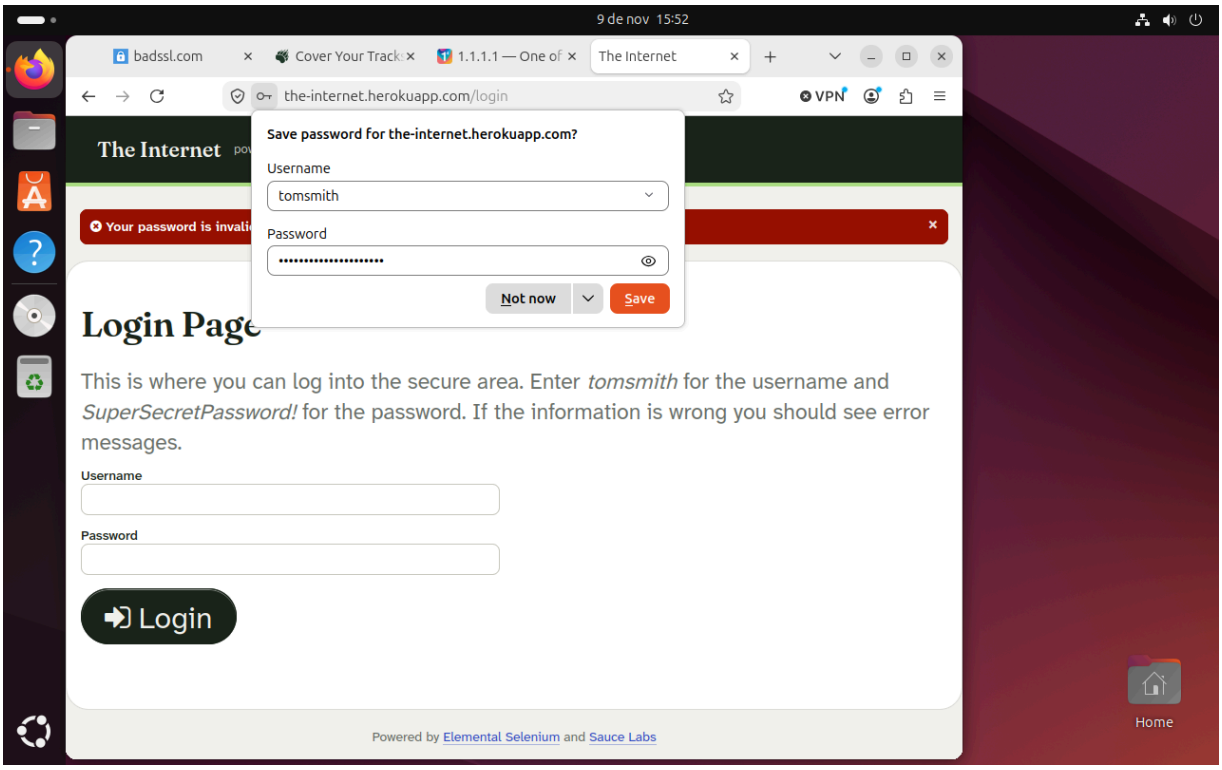


Figura 18: Login de prueba antes: Firefox ofrece guardar la contraseña.

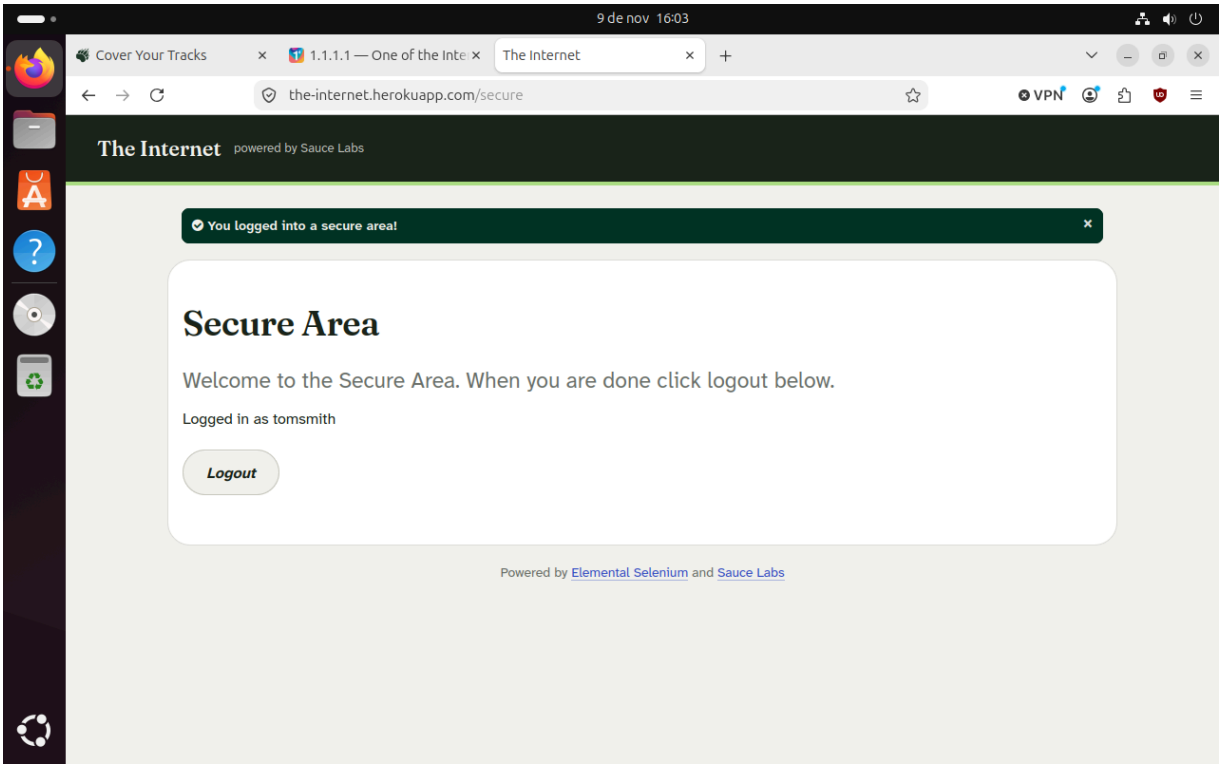


Figura 19: Login de prueba después: inicia sesión sin ofrecer guardarla.

A.3 Móvil

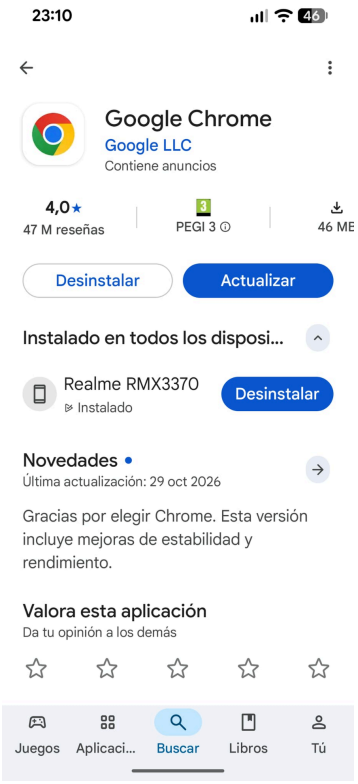


Figura 20: Pixel 10: Chrome con actualización pendiente en Play Store (A3).

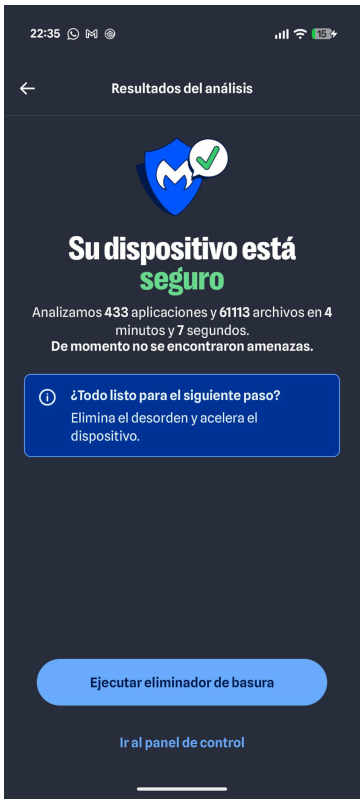


Figura 21: Pixel 9 Pro: escaneo de Malwarebytes sin amenazas (B1).

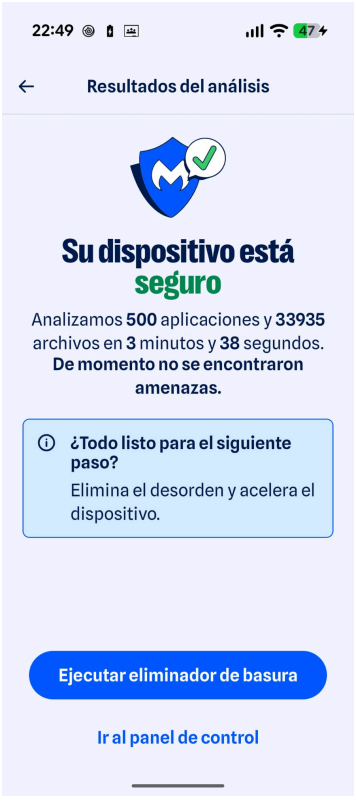


Figura 22: Pixel 10: escaneo de Malwarebytes sin amenazas (B1).

A.4 Web

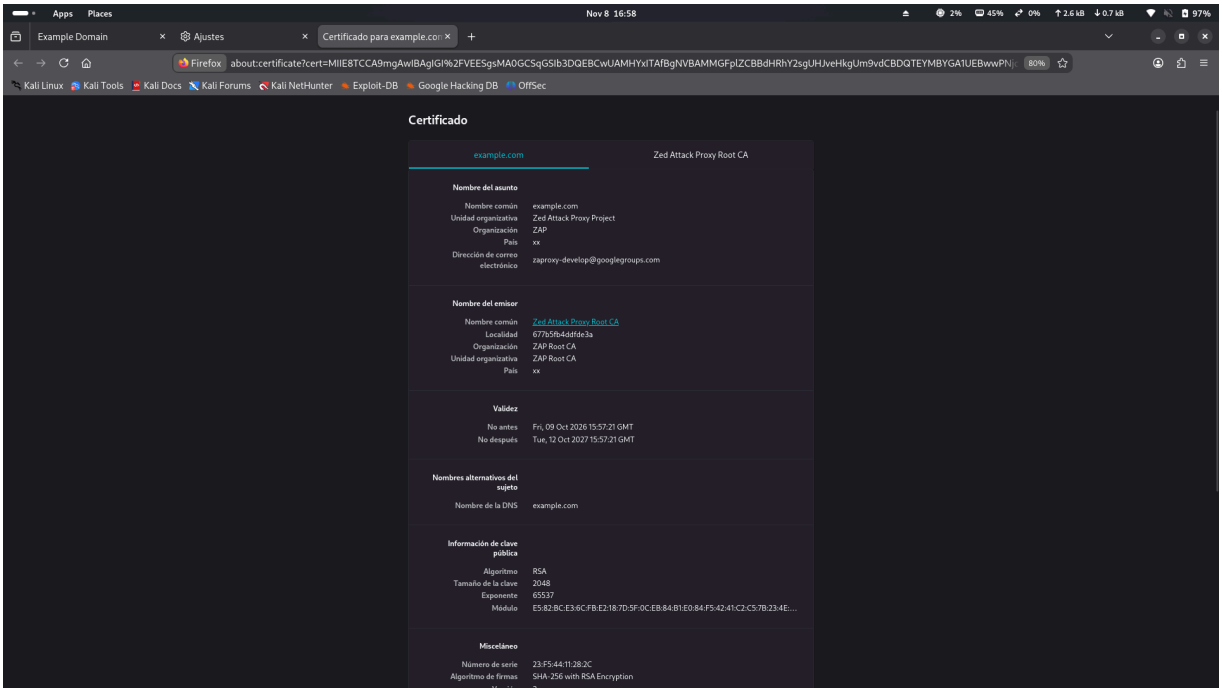


Figura 23: Certificado de example.com emitido por Zed Attack Proxy Root CA (trazabilidad HTTPS).

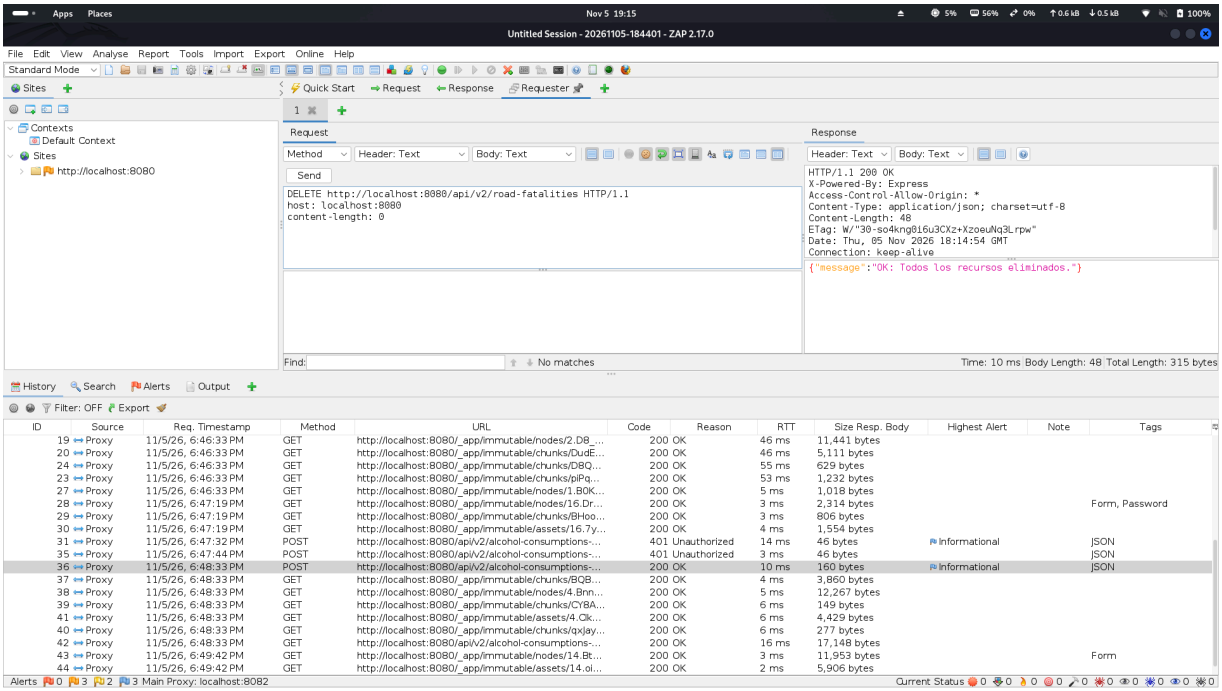


Figura 24: **DELETE** sin autenticación aceptado (vulnerabilidad 1).

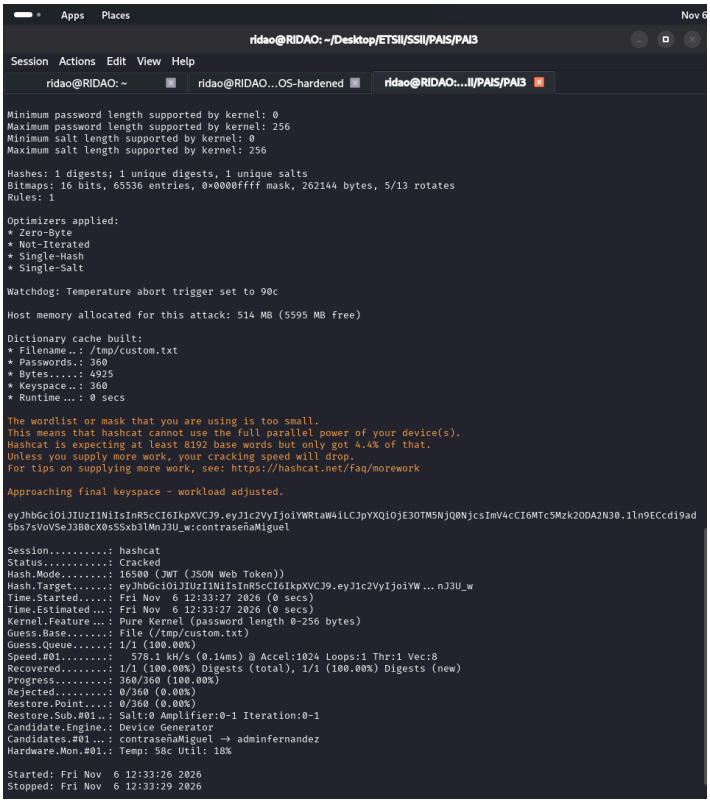


Figura 25: Secreto JWT crackeado con hashcat (vulnerabilidad 2).

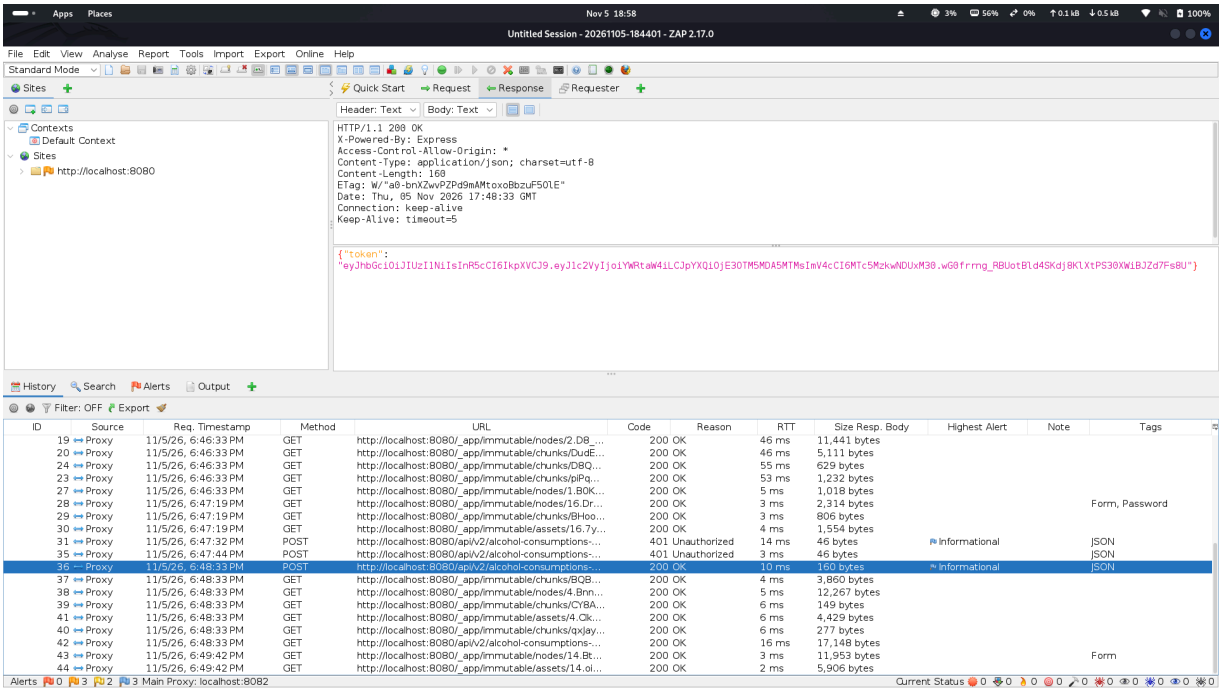


Figura 26: Login aceptado con **admin/admin** (vulnerabilidad 3).

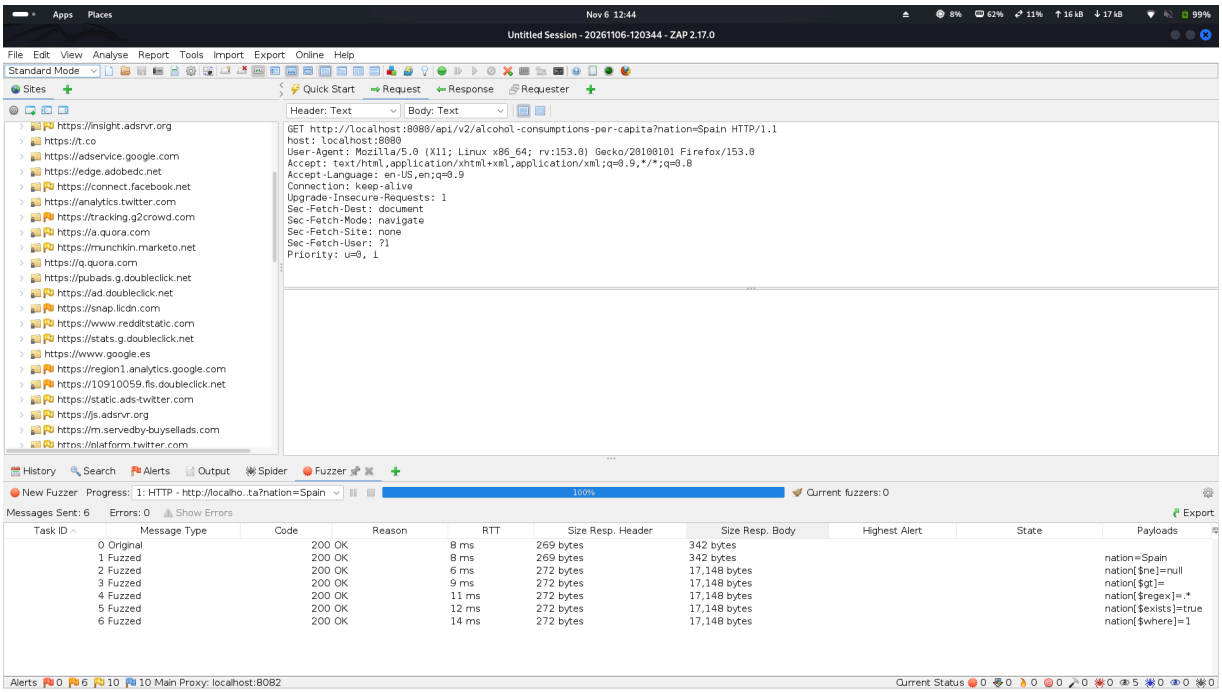


Figura 27: Fuzzer de ZAP con los 5 operadores NoSQL inyectados (vulnerabilidad 4).

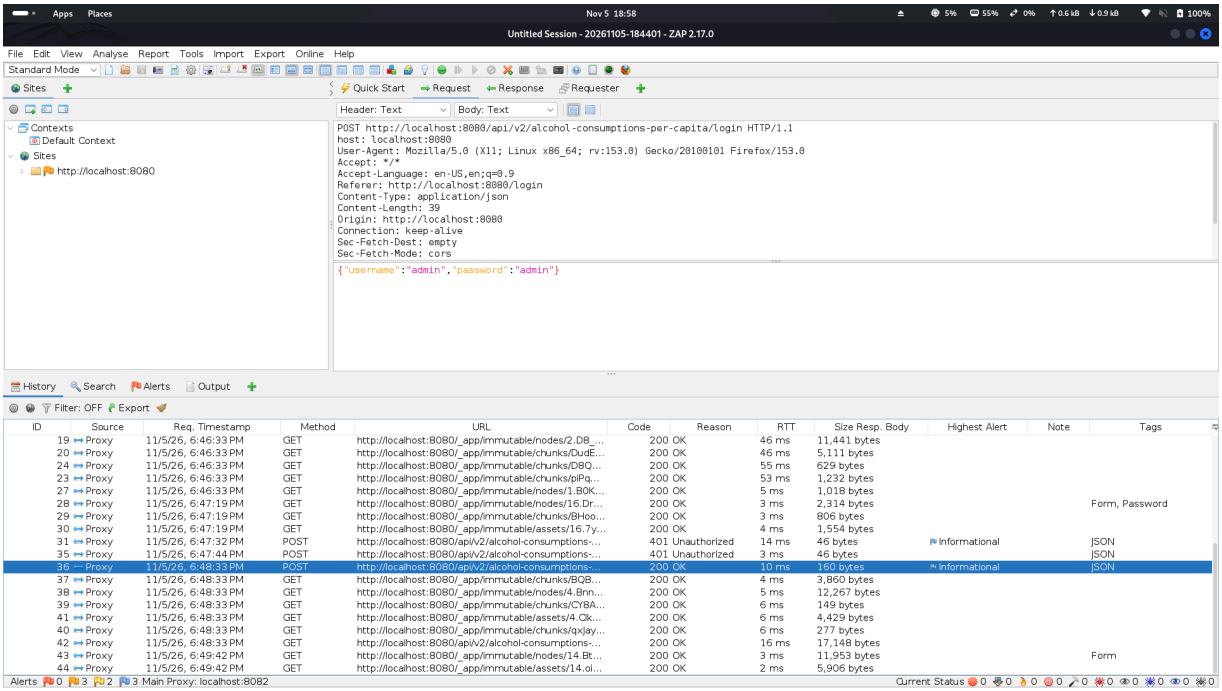


Figura 28: Pestaña History de ZAP con la petición de login (**POST**) y sus cabeceras, sin modificar (trazabilidad HTTP).

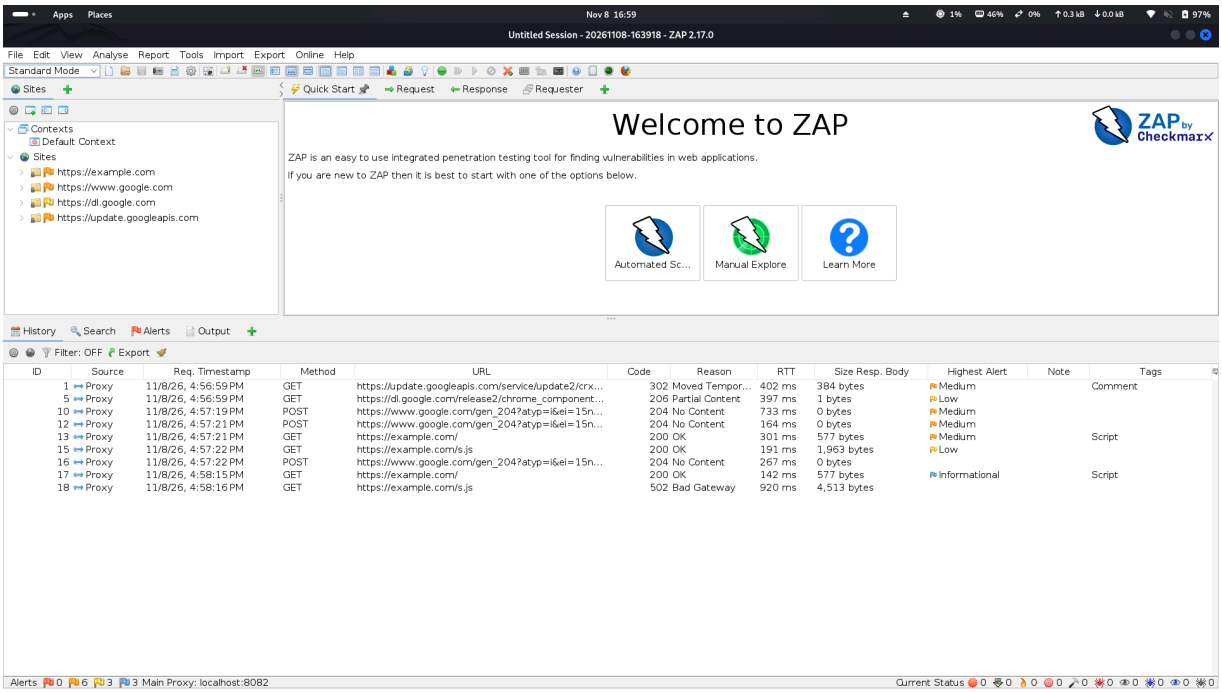


Figura 29: Pestaña History de ZAP con la petición HTTPS a example.com (**200 OK**).

Anexo B. Ficheros de evidencias

Todas las evidencias están en la carpeta **evidencias/** del zip, junto con los scripts y configuraciones usados:

- **sistemas/**: **hardening.sh**, **run_lynis.sh**, **sysctl-hardening.conf** y **custom.prf**.
- **evidencias/sistemas/Ubuntu/**, **Debian/** y **Kali/**: logs de Lynis (**lynis-*.log**), datos del informe (**report-*.dat**) y capturas de cada pasada.
- **navegador/**: **firefox/policies.json** y **desplegar.sh**.
- **evidencias/navegador/**: capturas antes y después de cada prueba.
- **movil/**: scripts **auditar_android.sh** y **conexiones_b2.sh**, y **checklist.md** con el antes y el después de cada móvil.
- **evidencias/movil/**: volcados de ADB antes y después de cada móvil, y capturas de Malwarebytes y de los ajustes revisados a mano.
- **web/**: lista de payloads usada en el Fuzzer (**payloads/nosql.txt**) y parche de la mitigación (**mitigacion-sos-hardened.patch**).
- **evidencias/web/**: informe HTML de ZAP, exportación del historial de ZAP (**zap-history-export.csv**), log de **npm audit**, log de las pruebas de XSS y Path Traversal, log de la verificación antes/después de la mitigación, notas de la sesión y capturas numeradas por apartado (**capturas/#1-*** a **#4-*** por vulnerabilidad y **#5-*** de la trazabilidad).

Anexo C. Sugerencias de Lynis que no se aplicaron

Sugerencia	Motivo
FILE-6310: **/home**, **/var** y **/tmp** en particiones separadas	Exige reparticionar el disco de un sistema ya instalado, con riesgo de perder datos. Se recomienda al instalar equipos nuevos.
BOOT-5122: contraseña en GRUB	Solo protege frente a quien tiene acceso físico al equipo, y sin cifrado de disco ese atacante puede leer el disco igualmente. Se recomienda junto con el cifrado de disco en portátiles.
SSH-7408 (Ubuntu y Debian): endurecer la configuración de SSH	Son equipos de usuario que no deben ofrecer SSH; UFW ya bloquea el puerto 22 entrante.
HRDN-7222: restringir los compiladores a root	El PDI y el personal técnico compilan código en sus equipos; quitarles el compilador rompería su trabajo.
AUTH-9286 y AUTH-9282: edad mínima de contraseña y caducidad de cuentas	Se aplicó la caducidad de 90 días, que es la medida principal. La edad mínima solo evita que el usuario vuelva enseguida a su contraseña anterior.
NETW-3200: protocolos **dccp**, **sctp**, **rds** y **tipc**	No se usan y se podrían bloquear igual que los módulos USB. Se deja como siguiente paso para no ampliar el alcance del script.
ACCT-9630, ACCT-9628 y ACCT-9626: auditd sin reglas (en Kali, sin activar) y sysstat sin activar	Están instalados pero sin configurar, igual que AIDE (ver Conclusiones).
LOGG-2154: enviar los logs a un servidor externo	Necesita un servidor central de logs, que no existe en este entorno.

Anexo D. Correspondencia con los estándares

Parte	NIST SP 800-115	CIS Benchmarks / OWASP	ENS, categoría Alta (RD 311/2022)
1. Sistemas	Revisión de configuración con Lynis antes y después	CIS Benchmarks de Linux: cortafuegos con política de denegar lo entrante, parámetros de kernel, módulos USB, contraseñas	op.exp.2 y op.exp.3 (configuración de seguridad), op.exp.8 (registro de la actividad: auditd, acct), op.exp.6 (protección frente a código dañino: rkhunter)
2. Navegador	Revisión de configuración con **about:policies** y pruebas antes y después	Configuración segura del navegador mediante política corporativa	op.exp.2 (configuración de seguridad), mp.com (protección de las comunicaciones: HTTPS obligatorio, TLS 1.2 mínimo, DNS cifrado)
3. Móvil	Revisión de configuración por ADB y escaneo con Malwarebytes	Política BYOD del enunciado (puntos A1-A5 y B1-B3)	op.exp.4 (mantenimiento y actualizaciones), op.exp.6 (código dañino), mp.com (redes inalámbricas seguras)
4. Web	Pruebas de penetración en caja negra con ZAP y verificación tras corregir	OWASP Top 10:2021 (A01, A02, A03, A05, A06, A07)	mp.s.2 (protección de servicios y aplicaciones web), op.acc (control de acceso y autenticación), mp.sw.2 (pruebas de seguridad antes de la puesta en producción)